

ZIREN: Succinct Arguments for MIPS32 Execution on GPUs*

Stephen Duan^{†1, 2}, Jingping Yin¹, Blake Hou¹, Vangher Pham¹, Ethan Zhu², and Ming Guo¹

¹ZKM Team

²GOAT Network

September 2026

Abstract

ZIREN is a zero-knowledge virtual machine for MIPS32 (Release 2) execution and the second generation of zkMIPS [ZKM24], which was the first zkVM for the MIPS32 instruction set itself rather than for a MIPS-like model. It proves the execution of arbitrary MIPS32 programs, and the architecture rewards that choice: Release 2 is finished, so an arithmetisation pinned to it never chases a moving instruction set and the verifying keys derived from it stay valid; its fixed-width encodings in a handful of formats let one row of one unit carry an instruction’s whole frame, with no central control table; and every compiler that already targets MIPS32 reaches the guest without a bespoke backend, so Rust, Go and C programs become the same ELF. Its delayed branches make a row’s successor data rather than the following address, which shapes everything above.

Four layers carry an execution to a proof: an instruction frame per row, whose state bus balances only if the rows form one chain; one offline-memory-checking argument for registers and memory, carried across shards by an elliptic-curve digest, with one lookup argument and one zerocheck per shard; a jagged reduction of tens of thousands of columns of widely differing heights to one dense-polynomial evaluation that a single batched WHIR proof opens in the unique-decoding regime; and a recursion layer whose verifying keys are enumerated from the machine’s declared shapes rather than collected from runs.

Soundness is composed by a union bound over each profile’s two dozen transcript components rather than quoted as the smallest of them: 100.70 bits for a core shard proof, 100.76 for a recursion node, 100.83 for the outer wrap, 99.18 across the three, and 93.50 once every node of a measured block’s tree is charged. All of these bound the *interactive* protocol: Fiat–Shamir subtracts $\log_2 Q$ against an adversary making Q oracle queries, and the cross-shard digest is an assumption rather than an error term. An executable Lean 4 ISA model and extracted determinism theorems close 17 of 106 cases and identify a genuine under-determination at the syscall bus. One NVIDIA RTX 5090 proves a 288-million-cycle Ethereum block at 5.9 MHz and four reach 20 MHz on 420 million cycles, with each engineering change measured against the configuration it replaced; those timings precede a correction to the Poseidon2 round schedule, and §4 states which claim attaches to which revision.

Index terms: arithmetic circuits, formal verification, GPU acceleration, MIPS32, polynomial commitment schemes, round-by-round soundness, verifiable computation, zero-knowledge virtual machines.

1 Introduction

A *succinct argument for machine execution* lets a prover convince a verifier that a program, run on a given instruction-set architecture, produced a given output, with a proof whose verification cost is essentially independent of the length of the run. Representative RISC-V systems include SP1, OpenVM, ZisK and Jolt [Suc24, Axi24, Zis25, AST24]; we restrict system-level comparisons to these four. Such systems are commonly called zero-knowledge virtual machines (zkVMs), although the term does not by itself establish that a proof hides its witness, and the proofs studied here, including ours, are succinct but not hiding.

*Source and artifacts: <https://github.com/ProjectZKM/Ziren>, v2.0.0.

[†]Corresponding author: stephen.d@zkm.io

Section 1.1 states the public relation, and Theorem 8.1 the conditions under which the implemented protocol is knowledge sound for it.

The benchmark used throughout is an Ethereum block of several hundred million instructions [Eth26b]. Public block-proving workloads spend much of their work on hashing, for Merkle updates, signatures and content addressing, because a hash commits to data the verifier does not hold; we use the block as a benchmark rather than as the application.

MIPS32, and this project’s part in it. zkMIPS, the first generation of this system, was the first zkVM for the MIPS32 instruction set; earlier proof systems addressed MIPS-*like* models or named MIPS as one mode among several (§1.4). Three properties of the architecture itself pay for that choice, and none of them is about any one application. *It is finished.* Since its owner moved its own roadmap to RISC-V in 2021 [Tur21, Har25] the instruction set does not change, so the specification an argument system implements is fixed, and so are the verifying keys derived from it, since an extension ratified next year cannot invalidate this year’s arithmetisation or the artifacts released against it. *It is uniform.* Fixed 32-bit encodings in a handful of formats let one row of one unit carry an instruction’s whole frame, with no central control table to decode into and no compressed encodings to widen it. *It is tooled, and that is what carries languages.* Every compiler that already targets MIPS32 reaches the guest without a bespoke backend: Rust, Go and C programs compile to the same MIPS32 little-endian ELF, for which this repository carries the guest runtimes, and the architecture is deployed at a scale few others match [MIP26]. What follows from those properties, rather than motivating them, is that the guest software the fault-proof infrastructure of Ethereum rollups executes, namely Optimism’s Cannon and Kona [Opt23, OP 24], is in range for this system too, subject to compatible runtime conventions and image formats. We therefore ask:

Can the MIPS32 instruction set be given a succinct argument system whose prover cost is attributable to identified arithmetisation and scheduling choices on commodity GPUs, whose soundness is accounted for component by component rather than as a headline number, and whose constraint system is carried by a verification workflow with machine-checked theorem coverage?

Four properties of the problem make that hard, and they are what the design answers. First, MIPS32 executes the instruction after a branch before control transfers, so an instruction’s successor is data rather than the following address, and a machine sequenced by one central table indexed by cycle has to carry that data through the table. Second, the relations that hold such a machine together are multiset relations *across* units, not polynomial identities inside one, so discharging them unit by unit would cost one argument per bus per unit. Third, a shard of the block workload commits about $3.6 \cdot 10^4$ columns whose heights run from 32 rows to millions, so neither one commitment per column nor padding every column to a common height is affordable. Fourth, soundness of the argument yields a satisfying trace and nothing more: whether that trace is an execution of the advertised machine depends on the arithmetisation admitting one behaviour per input, which the proof system cannot establish about itself.

MIPS proof systems predate ZIREN. Zilch implements zMIPS, a MIPS-like processor model [MT21], and o1VM documents a prover with MIPS and RISC-V modes [BL24]. We cite them to establish the MIPS antecedents, not as systems whose design axes are contrasted below. The first generation of ZIREN, zkMIPS [ZKM24], uses a single CPU table over the Goldilocks field. Cannon [Opt23] executes MIPS inside a fault-proof game but adjudicates one instruction at a time on-chain rather than proving an execution succinctly. This paper describes the second generation of ZIREN [The25] as an implemented verifiable-execution system: its guest executor produces a MIPS32 execution record; opcode-specific tables and polynomial identities arithmetise that record; Jagged-WHIR commitments and recursion turn it into a succinct proof; and standalone verifiers check the resulting artifact. Section 4 fixes the implementation and evaluation boundary. Trace cells, proved guest cycles per second and proof bytes are proof-system metrics rather than silicon area, clock frequency or energy.

Contributions. The paper introduces no new cryptographic primitive. It describes how a MIPS32 machine is arithmetised, committed, opened and composed, and what that costs in prover time and in soundness. Where a component is standard, or is taken from an existing system, the section that builds it says so and cites it. What follows is what this paper adds: three constructions, and one discipline in how a result is reported.

The *instruction frame* (§6.2) arithmetises MIPS32 without a central control table: each opcode unit fetches its own instruction, performs its accesses at four fixed sub-cycle positions and passes the machine state on a bus, so a cycle costs one row of one unit. The architectural delay slot is what separates this from carrying the same template to another register machine. The instruction after a branch executes before control transfers, so a row’s successor is data rather than the following address, and the frame carries it as such. Lemma 6.1 is what the construction buys: the state bus balances only if the rows form one chain. This project’s first generation proved the same ISA with a control table (§1.4), so what is claimed is the arithmetisation and not the choice of machine.

Enumerated leaf keys (§9) derive the verifying-key allowlist from the machine’s declared shard shapes rather than collecting the keys that happened to appear. Quantising the committed geometry into buckets, and making the padding-column count a function of the bucket alone, renders the set of leaf programs enumerable: about $6.4 \cdot 10^3$ keys, generated without executing a guest. The alternative, which this system used until recently, is to prove a corpus and collect the keys that appear, which covers the workload sampled and leaves the next shard shape one uncollected key away from a refusal.

The *Jagged-WHIR composition* (§7, §8) opens the published jagged reduction with WHIR rather than BaseFold, under the concrete **KoalaBear** schedule of Table 3 and with a recursive verifier for it. Both components are published and the neighbourhood is populated (§1.4); the claim is the composition, its schedule and its verifier, not either part.

Component-level soundness accounting (§8.4) is the discipline rather than a construction: a union bound over components is standard machinery. What this paper does is carry it through, stating the bound per profile and again with every node of a block’s tree charged, rather than reporting the smallest component or a project target. The result is a smaller number than a headline would carry, which is the point of reporting it that way.

The supporting evidence is reported with its gaps: an executable ISA model checked against an independent emulator on 770 vectors, extracted determinism statements with their coverage, and a negative control that rejects forged trace heights (§10); and an evaluation of one end-to-end CPU–GPU prover, on one workload class and one GPU model, that attributes cost to identified choices and measures what the changes bought (§11, with the per-change measurements in Appendix D).

Theorem 1.1 (Conditional argument statement, informal). *Let **Emulate** be the MIPS32 execution relation of Section 3. The implemented protocol over the 31-bit field **KoalaBear** consists of one lookup argument and one zerocheck per shard, a jagged multilinear commitment opened through WHIR, and a recursion tree. Supported honest executions within the configured resource limits that halt with exit code zero produce accepting proofs; the arithmetisation admits no other halt (§10). Suppose that every shard and recursion-node protocol satisfies the round-by-round knowledge-soundness premise of Theorem 8.1, that child extractors accept the transcripts produced by parent extractors, and that Assumption 6.4 holds. Then the recursion tree is knowledge sound for its satisfying witness rows in the random-oracle model, with the conditional error bound of Theorem 8.1. Identifying those rows with a unique **Emulate** execution additionally requires the unit-determinism and real-row-exhaustiveness premises of Theorem 6.5; Section 10 reports their current coverage.*

The construction is one reduction per layer. Every executed instruction is one row of the unit that implements its opcode, and the row carries an *instruction frame*, so there is no central control table; registers and memory share one offline multiset argument whose cross-shard part is an elliptic-curve digest; every bus is discharged by one logarithmic-derivative argument per shard and every polynomial identity by one zerocheck; the tens of thousands of columns of a shard, of widely differing heights, are reduced by the jagged sumcheck of Hemo et al. [HJR⁺25] to one evaluation of one dense polynomial, which a single batched WHIR opening [ACFY24b] authenticates against every root. Section 5 states that chain as a protocol over primitives, and the sections after it build each one.

Concrete efficiency. The argument is not merely asymptotic. The internal measurements are same-hardware and same-input: a per-unit area census with a (*row, interaction*) cost model, the effect of three arithmetisation changes and one configuration change (−9.4% together), and scaling from one to four GPUs (Section 11). One combined configuration change, the lookup circuit’s layer order together with the doubled shard size it makes fit, reduces time by 28% for the pair; the present experiment cannot decompose the two

factors. All timings predate the Poseidon2 round correction described in Section 11.7; they isolate engineering effects but are not measurements of the corrected release.

Beyond proving: verifying the execution semantics. Soundness of the argument says that an accepting proof implies a satisfying trace. Whether that trace is an execution of the advertised machine is a separate question. So is whether each unit’s constraint system admits exactly one behaviour per input. We therefore report a verification flow for these questions. Its central object is extracted from the constraint system itself, but its present theorem coverage is partial.

Theorem 1.2 (Unit determinism, informal). *For every unit U of the core machine there is a mechanically extracted Lean 4 theorem stating that two rows of U that satisfy its constraints and agree on their bus inputs agree on their bus outputs. Of the 106 such theorems, 17 are closed by a tactic that lifts the statement to bounded integer arithmetic and replays the propagation loop of Picus [PCW⁺23] inside the proof assistant; the remainder are open and classified in Figure 8. Together with the bus arguments, unit determinism implies that an accepting shard proof determines, up to permutation, every row reachable from the entry state along the state and memory buses (Theorem 6.5); that this exhausts the real rows is a property of the unit set, which we argue but do not prove.*

The flow found real defects: the syscall unit’s argument half-words were not determined by the reduced argument it received (because 2^{32} exceeds the field modulus), and its Linux selector had no input pinning it. Both were fixed by carrying the exact half-words and the selector on the syscall bus, after which the unit’s theorem closes (Section 10.4).

1.1 The statement being proved

Let **Emulate** be the deterministic MIPS32 execution relation of Section 3, the relation the emulator realises: given a program image P (code and initial data), an input stream in and a host-supplied *hint* stream hint , it produces an output stream out and an exit status e , or diverges. ZIREN proves the relation

$$\mathcal{R} = \{((\text{vk}, h), (P, \text{in}, \text{hint})) : \text{vk} = \text{Setup}(P) \wedge \exists \text{out} : \text{Emulate}(P, \text{in}, \text{hint}) = (\text{out}, 0) \wedge h = H(\text{out})\},$$

where **Setup** maps a program image to its verifying key, a Merkle commitment to the program ROM and the preprocessed tables of Section 6, and H is SHA-256 over the committed output. The program image, input stream and hint stream are witness components, while (vk, h) is the public statement. The verifying key pins the image. The relation admits only executions that halt with exit code zero, because that is the only halt the arithmetisation admits as a valid execution to prove: **Emulate** itself accepts a non-zero exit, and one generated test program does exactly that and cannot be proved (§10). The exit status is therefore not a free public input of $\mathcal{R}$. Because the protocol applies no blinding and makes no zero-knowledge claim, witness status alone does not imply confidentiality of the input or hint stream. Honest executions *that halt with exit code zero* produce accepting proofs; extraction of (in, hint) from an accepting prover is conditional on the hypotheses of Theorem 8.1 and on the trace-to-execution premises of Theorem 6.5.

1.2 Techniques

This section gives an overview of the design and of the arguments behind the two theorems; the sections that follow give the definitions and proofs. Figure 1 shows the proving pipeline; Protocol 5.1 states the shard argument, and Section 5 the argument system as a whole.

The four ideas that carry the design answer those four difficulties in the same order, and are developed in the sections named. The *instruction frame* (§6.2) removes the central control table this system’s first generation used. Each opcode unit fetches its own instruction, performs accesses at fixed sub-cycle positions and passes the machine state on a bus, so a cycle costs one row of one unit. *One lookup and one zerocheck* per shard (§6.8, §6.9) discharge every bus and polynomial identity, batched across units and sharing one opening per column. *Stacking and the jagged reduction* (§7) preserve separate roots for preprocessed and main data while reducing tens of thousands of columns to one logical multilinear evaluation and one batched opening. The verifier depends only on the committed-area class and fixed column geometry. Finally, *unit determinism* (§10.2) is the local property used by the conditional trace-uniqueness argument: two satisfying rows that agree on bus inputs must agree on bus outputs.

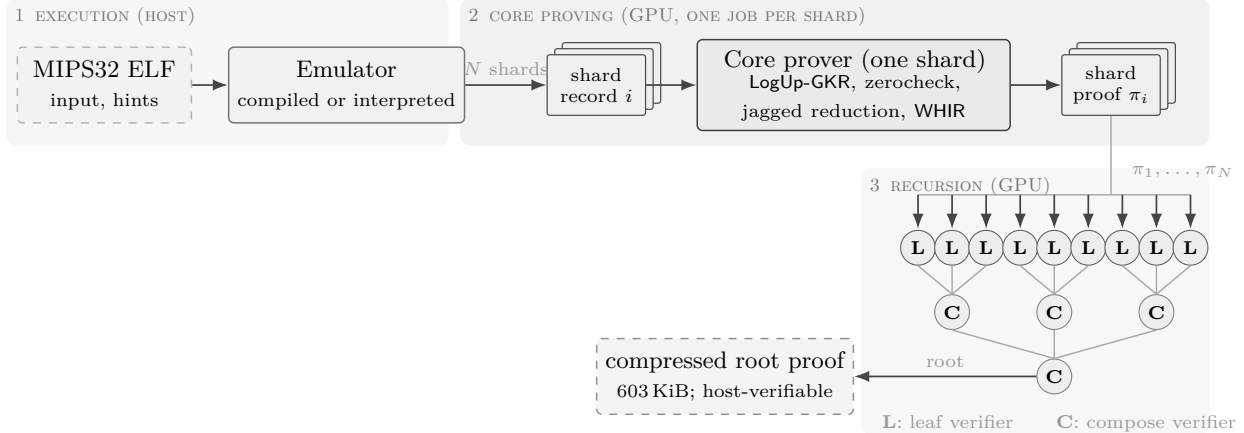


Figure 1: No stage of the pipeline sees the whole run: the proof of a block of any length is assembled from fixed-size pieces. The emulator cuts the run into N shards of a few million cycles and emits one record per shard; each record is proved independently as one Jagged-WHIR shard proof π_i ; leaf programs (L) verify one shard proof each and a range tree of compose programs (C, arity at most three) merges them; the root of the tree is the *compressed proof*, the object measured throughout this paper.

1.3 Scope and limitations

The proofs are not zero-knowledge. The soundness statement rests on a computational assumption for the cross-shard digest in addition to the information-theoretic bounds of the proximity test. The checked-in soundcalc report describes a schedule with 16 bits of lookup grinding, whereas the performance baseline uses none; Section 8.4 therefore does not assign the measured baseline that report’s numerical level. The determinism theorems cover the core machine and not the recursion machine, a substantial fraction remain open under the automation’s budget, and Theorem 6.5 is stated rather than machine-checked. The evaluation uses one workload class on one GPU model and predates the Poseidon2 round correction. The verifying-key allowlist is enumerated from the machine’s declared chip-set clusters, which are curated from the workloads it is built for rather than exhaustive over chip sets, so a guest that produces a shard outside them is refused rather than admitted (§9). Timing closure and power have no counterpart in this setting, and area is measured in trace cells rather than gates.

1.4 Related work

Table 1 places the systems discussed along the axes of this introduction: guest instruction set, field, arithmetisation, lookup argument, commitment with its proximity test, and the regime in which each project states its soundness level.

Chip-decomposed machines. SP1 [Suc24] established the template ZIREN follows: one unit per opcode, LogUp-GKR and recursive aggregation. SP1 Hypercube [Suc25a] added the jagged commitment and shard-level lookup and zerocheck shape; its just-in-time executor [Suc25b] is the model for ours. ZIREN inherits that template, the septic cross-shard digest and the shadow-read device that makes register accesses cheaper than memory accesses. It differs in the guest ISA and its frame, in opening the jagged commitment with WHIR rather than BaseFold, and in reporting a union bound over components rather than a single level; both systems count query soundness in the unique-decoding regime, and SP1 states that it relies on proximity-gap theorems rather than conjectures [Suc26] (retrieved 2026-09-24).

The closest layer-level neighbour. OpenVM [Axi24] likewise has no central control table; the instruction frame specialises that idea to MIPS.

Table 1: Design axes of ZIREN and four systems it is most often read against, from their public descriptions, read at the release named in the last column and retrieved on 2026-09-24. AIR is an algebraic intermediate representation, PCS a polynomial commitment scheme and IOPP an interactive oracle proof of proximity. The *Regime* column names the basis on which each project states its soundness level; regimes and conjectures differ and levels are not comparable without the accounting of Section 8.4. These projects move quickly, so every row is a dated snapshot rather than a standing description.

System	ISA	Field	Instruction tables	Lookup	Commitment / IOPP	Regime	Release
ZIREN	MIPS32	KoalaBear (2^{31})	unit per opcode, frames chip per opcode	LogUp-GKR	Jagged over WHIR	unique decoding, union bound	this paper
SP1 Hypercube [Suc25a]	RV64IM	KoalaBear		LogUp-GKR	Jagged over BaseFold	unique decoding, no proximity-gap conjecture	v6.8.0
ZisK [Zis25]	RV64IMA	Goldilocks	micro-operations, PIL2	LogUp	PIL2 (e)STARK	not stated	v1.3.0-alpha
OpenVM [Axi24]	own ISA, RV32IM transpiler	BabyBear	modular executor chips	LogUp	SWIRL: stacked WHIR	unique decoding at the leaves, list decoding above	v2.0.2
Jolt [AST24]	RV64IMAC	BN254 scalar (Dory), 128-bit (Akita)	Twist and Shout, small R1CS	Twist and Shout	Dory (default), Akita (lattice)	not stated	v0.3.0-alpha

The closest current overlap. OpenVM’s SWIRL combines stacked WHIR with LogUp-GKR interaction reduction, which is the same pair of ideas as this paper’s Jagged–WHIR layer. Four axes separate them at the snapshot above: we place columns of differing heights under ordered rounds whose committed areas, padding included, form one logical dense vector and reduce every column claim by the jagged sumcheck; our WHIR rounds fold several variables at a time under the per-round arity and rate schedule of Section 8; KoalaBear rather than BabyBear changes the extension degree available for challenges and hence the fingerprinting term; and our soundness level is a union bound over components rather than a project target, which is a different basis and not numerically comparable.

And a second. Ceno is a second point of proximity: its backend selects a commitment at run time, offering a jagged reduction over BaseFold by default and WHIR as an alternative, rather than the two in composition [LZZ⁺24] (repository revision 7c58fdd4, retrieved 2026-09-24). The composition presented here therefore lies adjacent to published work, and what is claimed is the composition, its schedule and its recursive verifier, not either constituent. Where the axes of the table are design axes, no cross-system performance claim follows from them, and no same-hardware comparison against any of these systems is attempted here. Machine-checked constraint-system correctness is an axis the table omits deliberately: stating where other projects stand would need a dated survey of each project’s artifacts at a fixed revision, which we have not performed, so we report our own coverage (Section 10) and make no comparative claim.

Where the prover’s work is put. Jolt [AST24] evaluates instructions by lookups rather than by per-opcode constraints, moving the dominant cost into the lookup argument and its multiplicities; since v0.2.0 it does so through the Twist and Shout arguments [ST25] in place of the Lasso construction [STW24] of its original design, with a small R1CS instance for instruction fetch. ZIREN uses lookups for byte-granular facts and for buses, and evaluates instructions by polynomial constraints. ZisK [Zis25] instead lowers the RV64IMA execution to micro-operations constrained in PIL2 and commits a STARK trace. The evaluation of Section 11 measures this system against itself.

Systems not contrasted above. The wider field is larger than the table, and we name it rather than leave it implicit. RISC Zero [BGt23] proves a RISC-V machine through a control table and a FRI-based STARK with continuations. Cairo [Sta21] and Valida [Lit24] take the opposite route to ZIREN on the ISA question, designing an instruction set for proving instead of inheriting a hardware one. Ceno [LZZ⁺24] builds non-uniform circuits per basic block and proves them with GKR over parallel segments. Nexus [Nex24]

accumulates execution by folding rather than merging shard proofs in a tree. *Airbender* [Mat25] targets Mersenne-31 with a degree-two arithmetisation and DEEP-FRI, a different point on the trade-off of Section 2. *Cysic* [Cys25] pursues dedicated proving hardware, outside the CPU–GPU scope of Section 4. None is measured here.

Fields, proximity and commitment. The field, Poseidon2 permutation [GKS23], constraint interfaces and transforms come from Plonky3 [Pol24]. Circle STARKs [HLP24] reach smooth Mersenne-31 domains through the circle group. *KoalaBear* instead has two-adicity 24 and uses ordinary Reed–Solomon domains, requiring the word splitting of Section 2; *Binius* [DP23, DP24] avoids that split with binary towers. The proximity line runs from FRI [BSBHR18] through DEEP-FRI [BSGKS20] and proximity-gap bounds [BSCI⁺20] to BaseFold [ZCF23], STIR [ACFY24a] and WHIR [ACFY24b]. *Ligero* [AHIV17] and *Brakedown* [GLS⁺21] are linear-time alternatives not used here. The jagged commitment is from [HJR⁺25], with the branching-program evaluation of Holmgren and Rothblum [HR18]; our contribution at this layer is its composition with WHIR and the measured schedule. The lookup uses Haböck’s logarithmic derivative [Hab22] in its published GKR realisation [PH23]. Plookup [GW20] and cq [EFG22] are univariate alternatives that do not transfer directly to this multilinear many-bus workload. The memory argument descends from [BEG⁺94].

MIPS, embedded verification, and circuit verification. Zilch’s zMIPS verifies a MIPS-like instruction model and composes instruction proofs [MT21]; o1VM documents a segmented prover intended to run in MIPS and RISC-V modes [BL24]. ZIREN differs by targeting the MIPS32 execution relation used by the rollup fault-proof ecosystem, by its chip-decomposed AIR and Jagged–WHIR commitment, and by its recursion and verification collateral. Cannon [Opt23] is a single-step MIPS interpreter for interactive fault proofs, and its suite is one of our conformance inputs; Kona [OP 24] is the fault-proof program of the same ecosystem. zkMIPS [ZKM24] is the first-generation ZIREN prover. Proofs reach constrained devices today as authentication and attestation protocols [CJSC23, SD21, DDGM23, EH24], where the statement is small and about the device; a zkVM for the device’s instruction set instead makes its software execution the statement and the constrained party the verifier.

Picus [PCW⁺23] introduced the determinism formulation of under-constrained circuits and an SMT solver for it; Section 10.2 states the same property and discharges it in Lean 4, because we could not run the public solver at the unit sizes of Table 9. OpenVM’s verification project extracts the AIR of its RV32IM and precompile chips and proves refinement to Lean reference specifications [Ope26]; its closed opcode proofs provide a stronger correctness baseline than the partial determinism coverage reported here, while targeting a different ISA and constraint system. The accounting follows the round-by-round methodology of [ACFY24b] and is computed with *soundcalc* [Eth26a]; the ethSTARK documentation [Sta23] provides another published treatment of this style of accounting.

1.5 Organisation

Section 2 defines the field, the code, and the unit, bus and determinism notions the rest of the paper is stated over. Section 3 defines the execution relation, and Section 4 fixes what is measured and what is not. Section 5 states the argument system as a protocol over four primitives. Sections 6 to 8 instantiate them: Section 6 builds the arithmetisation and proves the bus lemmas and the trace-uniqueness theorem, Section 7 builds the jagged commitment, and Section 8 instantiates the opening and accounts for soundness component by component. Section 9 then composes shard proofs into a block proof and states what binds that composition to the advertised machine. Section 10 describes the verification flow and its findings, and Section 11 the concrete efficiency. Appendix A gives the just-in-time executor and Appendix E specifies the system interfaces, configuration parameters, integration paths and delivered verification artifacts.

2 Preliminaries

2.1 Field, polynomials and codes

Field. All traces live over the prime field $\mathbb{F} = \text{KoalaBear}$, $p = 2^{31} - 2^{24} + 1$. Its multiplicative group has two-adicity 24, which bounds the Reed–Solomon domains available to the proximity test: the rate schedule of

Section 8 is chosen so that the largest domain stays within that limit. Random challenges and all sumcheck arithmetic use the degree-four binomial extension $\mathbb{F}_{p^4} = \mathbb{F}[X]/(X^4 - 3)$; a sumcheck round polynomial of degree d therefore has soundness error $d/|\mathbb{F}_{p^4}| \approx d \cdot 2^{-124}$ per round. Note that $2^{32} > p$: a 32-bit machine word cannot be represented by a single field element, and every word in the arithmetisation is a vector of four field elements holding its little-endian bytes.

Multilinear extensions and sumcheck. For $f : \{0, 1\}^n \rightarrow \mathbb{F}$ the multilinear extension $\tilde{f} \in \mathbb{F}[X_1, \dots, X_n]$ is the unique multilinear polynomial agreeing with f on the hypercube, $\tilde{f}(\mathbf{z}) = \sum_{\mathbf{b}} f(\mathbf{b}) \text{eq}(\mathbf{b}, \mathbf{z})$ with $\text{eq}(\mathbf{b}, \mathbf{z}) = \prod_i (b_i z_i + (1 - b_i)(1 - z_i))$. The sumcheck protocol [LFKN92] reduces a claim $\sum_{\mathbf{b} \in \{0, 1\}^n} g(\mathbf{b}) = \sigma$ about a low-degree g to a claim $g(\mathbf{r}) = \sigma'$ at a random $\mathbf{r} \in \mathbb{F}_{p^4}^n$ in n rounds, each sending a univariate polynomial of degree $\deg g$. We write a claim about a multilinear $\tilde{f}$ as the pair $(\mathbf{z}, v)$ meaning $\tilde{f}(\mathbf{z}) = v$.

Reed–Solomon and constrained Reed–Solomon codes. For a smooth domain $L \subset \mathbb{F}^*$ (a coset of a subgroup of order 2^d) the code $\text{RS}[\mathbb{F}, L, m]$ is the set of functions $L \rightarrow \mathbb{F}$ that agree with a polynomial of degree $< 2^m$; its rate is $\rho = 2^m/|L|$. Identifying the coefficient vector of such a polynomial with a function on $\{0, 1\}^m$, a codeword is also the evaluation of an m -variate multilinear $\hat{f}$ at the points $(x, x^2, x^4, \dots, x^{2^{m-1}})$, $x \in L$. Following [ACFY24b], the *constrained* code $\text{CRS}[\mathbb{F}, L, m, \hat{w}, \sigma]$ is the subset of $\text{RS}[\mathbb{F}, L, m]$ whose multilinear satisfies $\sum_{\mathbf{b} \in \{0, 1\}^m} \hat{w}(\hat{f}(\mathbf{b}), \mathbf{b}) = \sigma$ for a weight polynomial $\hat{w}$; the weight $\hat{w}(Z, \mathbf{X}) = Z \cdot \text{eq}(\mathbf{X}, \mathbf{z})$ expresses the evaluation claim $\hat{f}(\mathbf{z}) = \sigma$. A *fold* of a codeword by a challenge α maps $f : L \rightarrow \mathbb{F}$ to $\text{Fold}(f, \alpha)(x^2) = \frac{f(x) + f(-x)}{2} + \alpha \frac{f(x) - f(-x)}{2x}$, which is a codeword of $\text{RS}[\mathbb{F}, L^{(2)}, m - 1]$ whenever f is a codeword, and equals the partial evaluation of $\hat{f}$ at $X_1 = \alpha$.

Hashing and transcripts. Merkle commitments and the Fiat–Shamir transcript [FS87] use the Poseidon2 permutation [GKS23] over $\mathbb{F}$ at width 16 with a 248-bit digest (eight field elements). Its S-box has degree three, so the round schedule is the one tabulated for a 31-bit prime at that width and degree, eight full and twenty partial rounds; a lower S-box degree needs *more* partial rounds for the same margin, and the count for degree seven is thirteen. The same permutation is used by the transcript, commitment trees and recursion machine, so one schedule fixes all three. As Table 12 records, the configuration measured in Section 11 carried the degree-seven count. Proof-of-work grinding of b bits means the prover searches for a nonce whose absorption yields b leading zero bits in the squeezed challenge; it multiplies the cost of the corresponding cheating strategy by 2^b and appears additively in the security accounting.

2.2 Units, buses and determinism

Definition 2.1 (Unit). A *unit* (a *chip* in the implementation and in the literature this design descends from) is a table over $\mathbb{F}$ of a fixed width w together with a set of polynomial constraints of degree at most three in the w columns of one row, a distinguished boolean column is `_real` gating every constraint, and a set of *interactions*. A row is *satisfying* if it satisfies every constraint.

Definition 2.2 (Interaction and bus). An interaction of a unit is a triple $(\text{bus}, \mathbf{f}, m)$ where *bus* is a name, $\mathbf{f}$ is a tuple of linear expressions in the row’s columns and m is a linear expression in the columns (the multiplicity), marked as a *send* or a *receive*. Multiplicities are non-negative integers bounded well below p : on the sending side of the arithmetisation of Section 6 they are zero, one, or a value range-checked against the byte table, while the preprocessed **Program** and **Byte** tables receive with the number of times a row was used in the shard, which the shard’s cycle budget bounds by 2^{25} . In either case a sum over the rows of a shard stays far below p and is read as an integer. For a set of tables, the *multiset of a bus* is $\{(\mathbf{f}(r))\}$ with multiplicity $\sum m(r)$ over the sends of all rows r of all units, and likewise for the receives. A bus *balances* if its send and receive multisets are equal. Two distinct facts connect that equality to the fraction identity of Section 6.8 over $\mathbb{F}_{p^4}$, and they should not be conflated. The multiplicity bound is what stops the integer counters wrapping modulo p , so a balanced bus really is a statement about integer multiplicities rather than residues. It does *not* make the implication deterministic: the identity is checked after compressing each tuple with a sampled challenge and evaluating at further sampled points, so a bus that does not balance can still satisfy the identity, with probability bounded by Schwartz–Zippel in the number of interactions and the

degree of the compression. Multiset equality therefore *implies* the fraction identity unconditionally, while the converse holds up to that fingerprinting error, which Section 8.4 carries as a term of the soundness budget rather than treating as zero.

Lookups as fractions. A lookup argument proves that two multisets of field-element tuples agree. In the logarithmic-derivative form of Haböck and its GKR realisation [Hab22, PH23], written **LogUp-GKR** throughout, each tuple $\mathbf{f}$ on bus $\mathbf{bus}$ with multiplicity m contributes a fraction $m/(\alpha - \mathbf{bus} - \sum_{j \geq 1} \beta^j f_j)$ for random $\alpha, \beta \in \mathbb{F}_{p^4}$. Senders and receivers must sum to the same value, which a layered GKR circuit [GKR08] computes and proves by sumcheck. Section 6.8 proves all buses of a shard by one such argument.

Zerocheck. A *zerocheck* proves that a polynomial vanishes on a set of points: to show $g(\mathbf{b}) = 0$ for every $\mathbf{b}$ in a subcube, the verifier samples $\mathbf{r}$ and the prover runs a sumcheck on $\sum_{\mathbf{b}} \mathbf{eq}(\mathbf{b}, \mathbf{r}) g(\mathbf{b})$, which is zero for every $\mathbf{r}$ if and only if g vanishes on the subcube, up to the sumcheck’s error. Section 6.9 proves all polynomial constraints of a shard by one such check.

Definition 2.3 (Shard relation). A *shard* is a set of tables, one per unit, together with public values $\mathbf{pv}$ (the entry and exit state, the address cursors of the global memory tables and the septic digest, Section 3). It is *valid* if every real row is satisfying and every bus balances, where the public-values table contributes the entry state as a send and the exit state as a receive on the state bus, and the program bus is received by the preprocessed program table with the recorded multiplicities.

Definition 2.4 (Inputs, outputs and determinism of a unit). For a unit U , the *inputs* $I_U(r)$ of a row r are the values the row consumes from elsewhere: the tuples of its receives, the program tuple it fetches, and the previous-value part of its memory accesses. The *outputs* $O_U(r)$ are the values it produces for elsewhere: the tuples of its sends other than the fetch, and the new-value part of its memory accesses. Fetch and previous value are sends in the sense of Definition 2.2, because the preprocessed table and earlier access are their sources, but they are inputs of the row. Thus the direction of a bus interaction is not by itself the direction of the data. U is *deterministic* if for all satisfying real rows r, r' ,

$$I_U(r) = I_U(r') \Rightarrow O_U(r) = O_U(r').$$

This is the notion of Picus [PCW⁺23] adapted to the interaction language: whatever a row consumes from another table is an input, whatever it produces for another table is an output.

Cost metrics. Throughout, the *trace area* of a shard or a block is the sum over units of rows $\times$ columns of the committed main trace, in cells; the *(row, interaction) count* is the number of bus interactions summed over all rows, which is the number of leaves of the lookup circuit; and *throughput* is the number of proved guest cycles per second of wall-clock time. Section 11 relates the three to measured prover time.

3 The machine and its execution relation

3.1 MIPS32 as a guest

Throughout this paper MIPS32 means Release 2 of the specification, the revision this work targets and the last before Release 6 re-encoded the instruction set incompatibly. The guest machine is a 32-bit MIPS32 core with 32 integer registers (register 0 hard-wired to zero), the HI/LO pair written by multiplication and division, and a little-endian byte-addressed memory. Three features of the ISA shape the arithmetisation more than anything else.

Branch-delay slots. The instruction that follows a branch or jump is executed before control transfers. The machine state therefore carries a program counter *pair*: the address of the current instruction and the address of the one that follows it. A sequential instruction advances the pair as $(\mathbf{pc}, \mathbf{next_pc}) \mapsto (\mathbf{next_pc}, \mathbf{next_pc} + 4)$; a branch instruction sets $\mathbf{next_next_pc}$ to the target or to $\mathbf{next_pc} + 4$. Consequently the successor of a sequential instruction is *data*, not $\mathbf{pc} + 4$: when the instruction sits in a delay slot its successor is the branch target. Every instruction row carries $\mathbf{next_pc}$ as a witnessed column for this reason.

Unaligned and narrow accesses. LB, LBU, LH, LHU, SB, SH address individual bytes and half-words of a word, and LWL/LWR/SWL/SWR merge the left or right part of an unaligned word with a register. Memory is modelled at word granularity, so these instructions select and merge bytes of a word in-circuit; they get units of their own (Section 6.1).

Bit manipulation. EXT, INS, WSBH, SEB/SEH, ROTR, CLO/CLZ and the conditional moves are common in hashing and serialisation code. All of them are supported natively rather than lowered.

3.2 Syscalls and precompiles

Guest programs reach the host through SYSCALL. Besides input/output the syscall table exposes *precompiles*: SHA-256, Keccak-f[1600], the Poseidon2 permutation, curve arithmetic on secp256k1, secp256r1, BN254, BLS12-381 and Ed25519, field arithmetic for BN254 and BLS12-381, 256-bit modular and 256×2048 -bit multiplication. A precompile reads and writes guest memory directly; each invocation becomes one or more rows of a dedicated unit whose memory accesses are timestamped at the invoking cycle.

Hash functions. Hashing is the dominant primitive of the workloads this class of machine is asked to prove, and it motivates the accelerator interface. An unaccelerated guest computing SHA-256 or Keccak in MIPS instructions pays tens of instructions per round, each occupying an instruction-unit row, whereas an accelerator computes a permutation in a handful of wide rows. Three hash families are accelerated. SHA-256 is split into its message-schedule extension and compression rounds, with a control unit sequencing memory accesses. Keccak-f[1600] is a sponge unit with its own controller and, at 2637 columns, is one of the two widest units (Table 9). Poseidon2 is accelerated for guests that verify proofs of this system. The same permutation also implements Merkle commitments and the Fiat–Shamir transcript (Section 2) and appears in the recursion machine (Section 9). Hint data written by the host into untouched memory becomes part of the memory’s initial state (Section 6.3), so it remains subject to the argument’s range discipline.

3.3 Clock, shards and the execution record

Execution is divided into *shards*. Within a shard a clock clk advances by five per instruction; a syscall additionally advances it by a bounded number of ticks so that the memory accesses of a precompile fit strictly between the calling instruction and its successor. An instruction executes at clk and its memory and register accesses occupy the four *sub-cycle positions* $\text{clk} + 1, \dots, \text{clk} + 4$, one per operand role (Section 6.2), so a $(\text{shard}, \text{clk} + \text{pos})$ pair identifies every access uniquely and the memory argument timestamps an access by $\text{clk} + \text{pos}$. The clock is bounded by 2^{25} , which caps a shard at $2^{25}/5 \approx 6.71$ M cycles whatever the configured shard size says. The clock is witnessed as a 16-bit limb and a 10-bit high limb, the same decomposition the memory argument range-checks. A shard therefore closes on whichever of three bounds binds first: that fence; the configured shard size, a few million cycles, against which the executor compares $\text{clk} + \text{max_syscall_cycles}$ after scaling the configured figure by four, since it is given in cycles and the clock advances by five per instruction; or a unit’s height budget. Which one binds depends on the configuration, and Section 11 reports the shard counts that result from four of them.

At a shard boundary the emulator emits, for every memory word touched in the shard, the value and timestamp of its first and last access; these are the leaves of the cross-shard memory argument. The prover arithmetises a shard record containing per-unit event lists, the touched-address ledger and public values. Those public values contain the committed-output and deferred-proof digests, entry and exit program-counter pairs and clocks, exit code, shard indices, global-memory cursors and row counts, and the septic digest of Section 6.4. They are the interface through which consecutive shards and recursion layers are chained.

3.4 The execution relation

Write $\text{Emulate}(P, \text{in}, \text{hint}) = (\text{out}, e)$ for the relation this section defines: the machine of §3 started on the program image P with the two streams available to it, run until it halts, produces the output stream out and the exit status e . It is deterministic: no step reads a clock, a random source or any host state other than the streams.

Table 2: Implemented verifiable-system components and their evaluated quantities. None of the reported quantities is a silicon PPA metric.

System component	Implementation	Reported quantity
Guest execution	interpreter or JIT executor for MIPS32	guest cycles and execution rate
Execution relation	opcode-specific trace tables, identities and multiset buses	rows, columns, interactions and trace cells
Proof generation	lookup, zerocheck, Jagged-WHIR opening and recursion	proof bytes and soundness terms
Verification	standalone compressed-proof verifier or optional outer wrapper	acceptance result and verified public values
Deployment runtime	coordinator plus one or more GPU proving workers	wall-clock time and proved guest cycles/s

The relation is realised twice, and the two realisations must agree. The reference is an interpreter: it implements every instruction, every syscall and every error path, and it is the only one that emits the event record the prover consumes. A just-in-time compiler serves the two jobs that dominate wall-clock time and need no events, namely running a guest for its output alone and producing, per shard, the descriptor from which a worker replays that shard. Three checks connect them, none of which is a proof of equivalence for every program: the descriptors of a compiled run are byte-identical to an interpreted one on the tested corpus, the specification vectors of Section 10 run on both, and continuous integration compares digests of the emitted records. The difference that matters to the rest of the paper is rate, because a host feeding several accelerators is bound by the executor rather than by the accelerator (§11.2); Appendix A gives the mechanism.

4 What is implemented, and what is measured

The sections that follow describe the proof system from the inside. This section identifies the implemented system boundary, because that boundary determines what can be measured and what those measurements mean. The processor is represented by arithmetic constraints rather than by synthesisable RTL, and what is evaluated here is the proving and verification software stack running on CPUs and GPUs.

Four configurations, and which claim attaches to which. One warning belongs here rather than distributed through the sections that make the claims. No single revision of this system exhibits every property reported in this paper, and the paper does not pretend otherwise. The commit named on the title page is the one the construction is described from, and it carries the corrected Poseidon2 round schedule. The timings and proof sizes of Section 11 were taken on earlier revisions of the same branch, before that correction and with the lookup grinding set to zero. The soundness figures of §8.4 are for the checked-in configuration, which spends 16 bits on that grinding and which Section 11 therefore does not measure. And the determinism count of §10.2 is for the constraint system with the syscall-bus change of §10.4 applied, which two units separate from the system that was timed. Each section states its own configuration where it reports; the reason to say it once here is that a reader who takes a number from one section and a number from another is comparing revisions, not configurations of one revision.

ZIREN has three implementation layers. The execution layer runs a MIPS32 program and emits the witness record. The proof layer constrains, commits and recursively compresses that record. The deployment layer schedules the work on host processors and GPUs and exposes standalone and wrapped verifiers. Table 2 maps these layers to the quantities evaluated in this paper and prevents proof-system metrics from being conflated with silicon metrics.

Trace area belongs to the proof relation. The trace area of a unit is its column count times its row count, and both are fixed by the constraint system and the execution rather than by the machine that runs the prover. Trace area determines how many field elements the prover must commit and open, and therefore

contributes to proof cost. It is not measured in square millimetres and does not convert into die area. An arithmetisation change can nevertheless be stated once, in cells, and evaluated on different accelerators, which is why the per-unit census of Section 11 is given in cells before it is given in seconds.

Throughput belongs to the implementation–accelerator pair. Proved guest cycles per second is not a property of the execution relation alone. It is what one implementation achieves on a specified accelerator, and a different device, or more of them, changes the figure without altering a constraint. Three things follow. Scaling has to be reported across device counts rather than as a single number. Figures measured on other projects’ hardware are context, not comparison. And the transferable part of the performance analysis is the cost model, which predicts prover work from area and interaction count while the implementation and accelerator determine the constants.

Realisation boundary. This paper evaluates a software proof system deployed on a host processor with one to four GPUs. It reports no RTL synthesis, place-and-route, process node, timing closure, silicon area or energy measurement; every frequency figure is proved guest cycles per second and is not a hardware clock. Section 10 reports how the executor, constraint system and recursive verifier are checked, together with the coverage that remains incomplete. Consequently, the results support claims about an implemented verifiable-computation system, its proof cost and heterogeneous-system throughput, but not claims about a fabricated processor or FPGA implementation.

Appendix E collects the reference material a deployer needs: interfaces, variable parameters, integration modes and delivered verification artifacts.

5 The argument system

This section states what is proved and how, independently of how any component is built. A *shard argument* reduces every claim about one shard’s execution to a single evaluation of a single committed polynomial, and one opening authenticates it. A *block argument* composes shard arguments by recursion into one proof of the relation $\mathcal{R}$ of Section 1.1. Sections 6 to 9 instantiate each component named here, and Section 10 asks whether the instantiation admits only executions of the advertised machine.

5.1 Parameters

The protocol is stated over the following parameters, and is symbolic in all of them: a different choice gives a different instantiation of the same protocol. Numeric values are fixed where the component is built. Table 3 fixes the proximity-test schedule of the checked-in core and compress profiles, §7.5 the stacking height and the row cube, §2 the permutation schedule, §3 the clock fence and §9 the recursion arity. The measured baseline of Section 11 differs from the checked-in configuration in two of these parameters, and one of them is a security parameter: it sets the lookup grinding to zero, and it carries the degree-seven Poseidon2 round count rather than the corrected degree-three schedule (§2).

- The base field $\mathbb{F}$ and the extension $\mathbb{F}_{p^4}$ from which challenges are drawn. The extension degree bounds every soundness term of the form $\deg/|\mathbb{F}_{p^4}|$.
- The permutation used for the transcript and for the commitment trees, with its round counts, and the arity of those trees.
- The stacking height $H = 2^\ell$, the row cube 2^n , and the bucket rule that maps a round’s real area to its committed area (§7.5). These fix the committed geometry, and with it the set of recursion programs a prover can produce.
- For the proximity test: the initial rate ρ , the per-round folding factors, the query counts, the number of out-of-domain samples, the grinding bits spent on each query phase and on the bus challenges, and the regime in which the query soundness is counted.
- For recursion: the arity of a compose node and the height of the tree of admissible recursion keys (§9).

- For the machine: the configured shard size, the clock fence and the per-unit height budgets, whichever of which binds first closing a shard (§3).
- The guest-accelerator set, which fixes the precompile units a shard may contain and therefore the unit set itself (§E.2).
- For the cross-shard digest: the curve and its extension, the bound N_{int} on the interactions a block proof may contain, and the per-interaction multiplicity bound B (§6.5).

Two properties of the instantiation are worth stating with the parameters rather than leaving to a discussion. Query soundness is counted in the unique-decoding regime, where no conjecture is needed, rather than at the Johnson bound or towards capacity (§8.4). And no blinding is applied anywhere: the protocol is succinct, not hiding, and a proof reveals what its committed trace reveals.

5.2 Syntax and components

We state the shard argument and each primitive it uses as a tuple of algorithms, so that the protocol of §5.3 can be read against interfaces rather than against any one construction. Throughout, pp are the public parameters of §5.1 and τ is the transcript the Fiat–Shamir transform maintains.

A *shard argument* is a tuple $\text{SA} = (\text{Setup}, \text{Prove}, \text{Verify})$:

$\text{Setup}(\text{pp}, P) \rightarrow (\text{pk}, \text{vk})$: on input a program image P , commits to the program ROM and the preprocessed tables and outputs a proving key and a verifying key.

$\text{Prove}(\text{pk}, T) \rightarrow \pi$: on input the execution record T of one shard, outputs a shard proof.

$\text{Verify}(\text{vk}, \text{pv}, \pi) \rightarrow b$: on input the shard’s public values, outputs $b \in \{0, 1\}$.

It is built from four primitives. A *unit* is a table of columns over $\mathbb{F}$ with a constraint system and a set of bus interactions (Definitions 2.1 and 2.2), and a shard is a set of units with a shared public-value row; units are data rather than algorithms, so they carry no syntax here. The other three do.

A *bus argument* $\text{BA} = (\text{Prove}, \text{Verify})$ certifies that the multiset of tuples sent on each bus equals the multiset received:

$\text{BA.Prove}(\tau, \{U_i\}) \rightarrow (\pi_{\text{bus}}, \mathbf{z}, \{v_{i,j}\})$: outputs a proof together with the point $\mathbf{z}$ and the claimed evaluations $v_{i,j}$ of column j of unit i at $\mathbf{z}$.

$\text{BA.Verify}(\tau, \pi_{\text{bus}}) \rightarrow (\mathbf{z}, \{v_{i,j}\}) \vee \perp$: accepts and outputs the point and the claims it has reduced to, or rejects.

A *constraint argument* $\text{CA} = (\text{Prove}, \text{Verify})$ has the same shape and certifies instead that every unit’s constraints vanish on its real rows, reducing that claim to evaluations at the same point.

A *jagged commitment* $\text{JC} = (\text{Commit}, \text{Reduce}, \text{Open}, \text{VerifyOpen})$ carries columns of differing heights:

$\text{JC.Commit}(\text{pp}, (c_1, \dots, c_M)) \rightarrow (\text{com}, \text{st})$: on input M columns of arbitrary heights, places them into the committed geometry and outputs a commitment and prover state.

$\text{JC.Reduce}(\text{st}, \{(i, \mathbf{z}, v_i)\}) \rightarrow (\pi_{\text{jag}}, \mathbf{z}^*, v^*)$: reduces any set of column-evaluation claims to a single evaluation v^* of one dense multilinear polynomial at one point $\mathbf{z}^*$.

$\text{JC.Open}(\text{st}, \mathbf{z}^*) \rightarrow \pi_{\text{open}}$ and

$\text{JC.VerifyOpen}(\text{com}, \mathbf{z}^*, v^*, \pi_{\text{open}}) \rightarrow b$: authenticate that single evaluation against the commitment.

Only the last interface constrains the others: because every claim is reduced to one evaluation of one polynomial, the number of units, their widths and their heights are invisible to the opening, and a unit may be added, widened or removed without changing the layer above it.

5.3 The shard argument

Protocol 5.1 states the reduction. Each step ends with claims of the same kind it began with, evaluations of committed columns, so the steps compose without the verifier ever reading a column.

Protocol 5.1 (Shard argument). **Statement:** a verifying key committing to the program and the preprocessed tables, and the public values of one shard.

Witness: the shard’s execution record.

1. **Commit.** The prover arranges each unit’s events into rows, places the columns of each commitment round into one dense vector and commits to it. Round order is fixed and the per-unit dimensions enter the transcript, so the geometry is public before any challenge is drawn (Section 7).
2. **Buses.** The verifier draws the bus challenges. The prover runs the bus argument, which reduces every send and receive of every unit to column evaluations at one point (Section 6.8).
3. **Constraints.** The verifier draws the constraint challenges. The prover runs the constraint argument on the same point, so each column is opened once for both arguments (Section 6.9).
4. **Jagged reduction.** The column claims of the two previous steps, together with explicit zero claims for the gaps in the committed geometry, are batched and reduced to one evaluation of the dense polynomial (Section 7.2).
5. **Assist.** The verifier does not materialise the weight table the previous step reduced against. A second sumcheck over the public column boundaries certifies it, and any point-extension coordinates are sampled here, in an order that is itself part of the protocol (Section 7.3).
6. **Opening.** One batched opening authenticates that evaluation against every commitment of step 1 (Section 8).

Output: a shard proof whose public values carry the entry and exit machine state, the memory-address cursors, the digest of committed output and the exit code that the relation of Section 1.1 is stated over, and a digest of the interactions this shard leaves unbalanced.

From shards to a block. A shard argument proves one shard in isolation. Three things tie the shards of one execution together, and each is discharged where it arises rather than by the opening: the machine state chains through the public values, the memory-address cursors tile, and the interactions that cross a shard boundary accumulate into a digest that must cancel over the whole execution (Section 6.4). A recursion layer then verifies shard proofs inside the argument system itself and merges them, so that the verifier of a block reads one proof rather than one per shard (Section 9).

What is proved. Two statements carry the construction, and both are conditional in ways the paper makes explicit rather than hiding. Theorem 8.1 bounds a cheating prover for the whole tree by the number of random-oracle queries times the sum of the per-node errors, plus the digest term, under a per-node round-by-round premise and a hypothesis about composed extractors. Theorem 6.5 shows that two satisfying traces for the same program and public values agree up to permutation, given unit determinism and the real-row premises that Section 10 reports coverage for; identifying that trace with an execution of the machine needs the conformance evidence of the same section. Neither statement is about a particular unit set, a particular field or a particular proximity test; those are the instantiation, fixed where each component is built.

6 The arithmetisation

Figure 2 shows instruction units connected by buses to a program ROM, memory subsystem and lookup tables. For a unit in the sense of Definition 2.1, the column count is its datapath width and the row count is its activity for an execution. A *bus* is an interconnect on which units exchange tuples. The lookup argument of Section 6.8 acts as the wiring by proving that every tuple driven on a bus is consumed exactly once. The

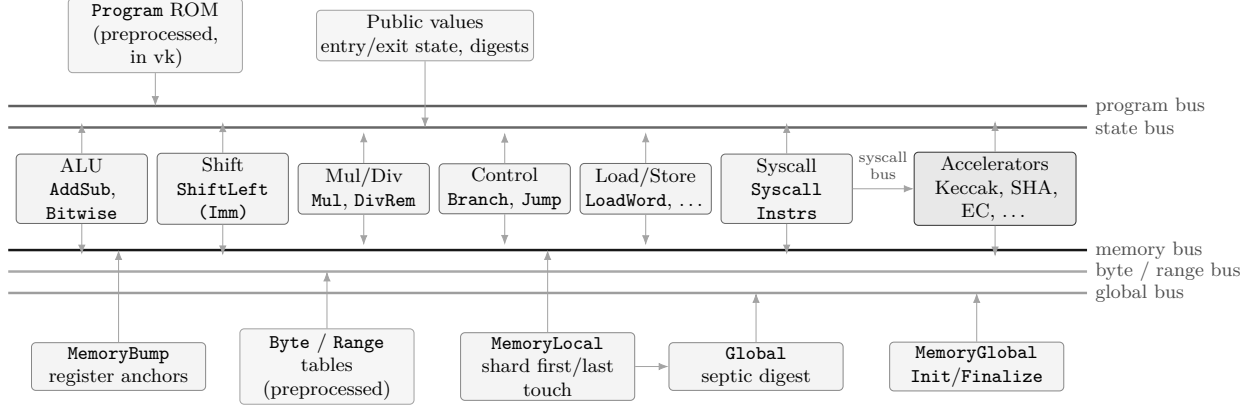


Figure 2: Each executed instruction occupies one opcode-unit row, and every relation between units crosses a named bus. The program and lookup tables are preprocessed; public values close the state chain and carry the cross-shard digests.

area of the design is the sum over units of rows $\times$ columns (*trace cells*); Section 11 reports it per unit, as a gate-count report would.

This section defines the constrained execution relation within the system boundary of Section 4. The unit widths determine its per-unit trace area (Appendix C), and the buses connect the units. One row per instruction, one shared memory argument, one lookup and one zerocheck fix the area measured in Section 11. The guest-accelerator set is configuration rather than fixed design: it determines which precompile units a shard may contain, and adding one changes the unit set and verifying keys (§E.2).

A shard is proved as a set of units. A unit is a fixed-width table over $\mathbb{F}$ with polynomial constraints of degree at most three on each row and interactions on named buses. Constraints are local to one row and evaluated at a random point of the multilinear domain (Section 6.9), so relations between rows are expressed as bus interactions. Every relation between units, or between different rows of one unit, is an interaction. This section describes the unit set, instruction frame, register and memory argument, lookup argument and zerocheck.

6.1 Unit set

Appendix C catalogues the units of the machine measured here, with their widths. Three families dominate the trace area of an Ethereum block: the memory-instruction units (*LoadWord*, *StoreWord*, *MemoryUnaligned*, *LoadNarrow*, *StoreNarrow*), the ALU units, and the *Global* unit that carries the cross-shard memory argument. The immediate forms of the ALU instructions (*ADDIU*, *ANDI*, *SLTI*, shifts by a constant, ...) have units separate from the register forms: an immediate row has one register read fewer and a narrower frame, and the split saves roughly a tenth of the width of those rows.

Row counts are rounded up to a multiple of 32 rather than to a power of two; the jagged commitment of Section 7 makes the padding cost proportional to the padding, not to the next power of two. Padding rows have *is_real* = 0; every constraint and every interaction multiplicity is gated so that padding rows are vacuous.

6.2 The instruction frame

A design organised around a central *CPU table*, as this system’s first generation was, has one row per executed cycle that fetches the instruction, performs the register accesses, chains the program counter and forwards a decoded instruction to an opcode unit over a bus; the opcode unit then spends a second row on the same instruction. The current design removes the central table: each instruction unit embeds an *instruction frame* and performs those duties itself, so one executed instruction is exactly one row of one unit. The frame consists of:

- **Program fetch.** The row sends $(pc, opcode, op_a, op_b, op_c, imm_b, imm_c)$ on the *program* bus; the preprocessed **Program** table receives it with the multiplicity recorded at trace generation. The instruction word therefore comes from the program image committed in the verifying key, and the unit's opcode selector is checked against it.
- **Register accesses.** Reads of op_b and op_c (unless immediate) and the read-modify-write of op_a at the sub-cycle positions 1..4 of the instruction's tick, through the register variant of the memory argument (Section 6.3). A write to register 0 is committed as zero: the frame carries an op_a_0 flag and pins the written word to zero when it is set, so units bind their result through a $(1 - op_a_0)$ factor rather than a gate.
- **State bus.** The row receives the tuple $(shard, clk, pc, next_pc)$ and sends the tuple $(shard, clk + \Delta, next_pc, next_next_pc)$, where the increment Δ is five for an ordinary instruction, one per sub-cycle position plus one, and a larger bounded value for a syscall, so that the accelerator's memory accesses fit between the calling instruction and its successor (Section 3); the public-values AIR sends the shard's entry state and receives its exit state. Lemma 6.1 is what this buys: the multiset balances only if the rows form a single chain, which is the shard's execution.
- **Clock.** clk is witnessed as a 16-bit and a 10-bit limb, both range checked, so that clock differences in the memory argument can be decomposed cheaply.

Lemma 6.1 (State-bus chain). *In a valid shard (Definition 2.3) with entry state s_0 and exit state s_N , the real instruction rows can be ordered $r_1, \dots, r_N$ so that r_i receives s_{i-1} and sends s_i , with $clk(s_i) > clk(s_{i-1})$; every real instruction row is in the chain, and N is the number of real instruction rows.*

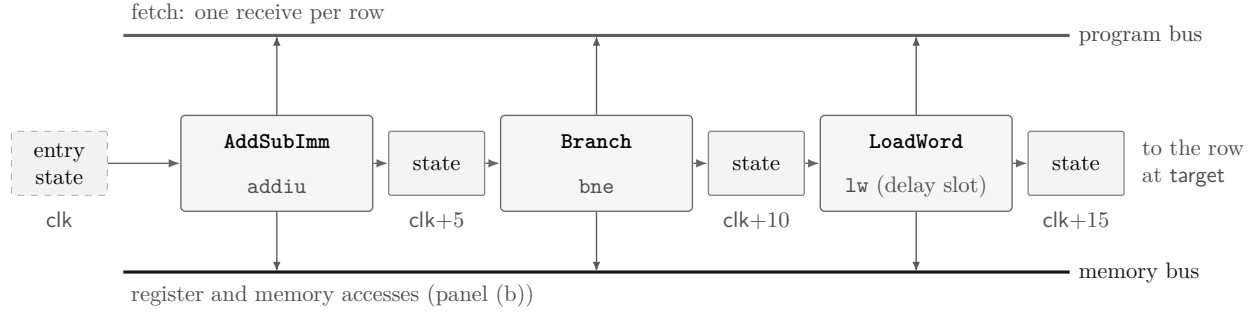
Proof. Each real instruction row receives exactly one state tuple and sends exactly one, with a strictly larger clock, and the public-values table sends s_0 and receives s_N . Balance of the state bus means the multiset of sent tuples equals the multiset of received tuples. Consider the directed graph whose vertices are the distinct state tuples and whose edges are the rows (from received tuple to sent tuple). Every vertex other than s_0 and s_N has in-degree equal to out-degree, s_0 has out-degree one more than in-degree and s_N the reverse, so the edge multiset decomposes into one $s_0 \rightarrow s_N$ path and cycles. A cycle is impossible because the clock strictly increases along every edge. Hence the rows form a single path from s_0 to s_N . $\square$

Figure 3 shows three consecutive rows and the buses that bind them. Three frame variants exist: the I-type frame (an immediate as a 4-byte word, used by memory and branch units), the R-type frame (two register reads), and a *shamt* frame for shifts by a constant, whose immediate is a single scalar. The frame byte-checks the word written to op_a ; this is the single place where a register value is range checked, and it is what lets memory-instruction rows omit byte checks on the words they move (Section 6.3).

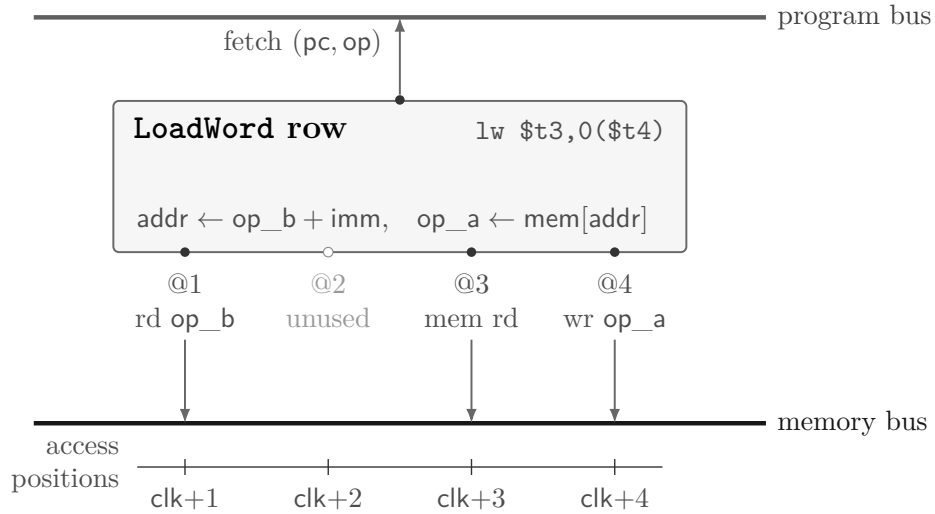
6.3 Registers and memory

Registers and memory share one offline memory-checking argument [BEG⁺94] in the multiset style used by SP1 [Suc24]: the state of address a is the tuple $(shard, clk, a, value)$ of its last access. An access at time (s, c) sends the previous tuple (s', c', a, v') and receives the new tuple (s, c, a, v) on the *memory* bus, with $v = v'$ for a read. The multiset balances if and only if every access consumes exactly the tuple produced by the previous access to the same address, provided timestamps strictly increase along each address's chain; the AIR of every access enforces $(s', c') < (s, c)$ by witnessing $c - c' - 1$, or $s - s' - 1$ when the shards differ, in range-checked limbs that cover a $clk + pos$ timestamp under a 25-bit clock fence, as a 16-bit limb and a 10-bit high limb (a one-bit `compare_clk` selector chooses the branch). Words are four byte columns; the argument carries them as such. Figure 4 follows one address through two shards: which of its tuples the memory bus balances, which are left to the cross-shard digest, and where the chain begins and ends.

Lemma 6.2 (Memory argument). *In a valid shard, for every address a the accesses to a form a chain ordered by $(shard, clk)$ in which each access consumes exactly the tuple produced by the previous access to a , starting from the tuple supplied by the initial-state row of a and ending at the tuple consumed by its final-state row; in particular every read returns the value of the last write.*



(a) The state chain across three consecutive cycles. Each row receives the machine state of its cycle from the register on its left and sends that of the next to the register on its right, fetches its instruction from the preprocessed **Program** ROM on the program bus, and places its accesses on the memory bus.



(b) One row enlarged: the four fixed sub-cycle access positions. Filled pins are accesses the opcode uses, hollow pins are unused.

Figure 3: The instruction frame binds consecutive opcode rows into one state chain (panel a) while assigning register and memory accesses to fixed sub-cycle positions (panel b). The state tuple is $(\text{shard}, \text{clk}, \text{pc}, \text{next_pc})$; the program tuple received from the ROM is $(\text{pc}, \text{opcode}, \text{op_a}, \text{op_b}, \text{op_c}, \text{imm}, \dots)$; each memory access sends the previous tuple $(\text{shard}, \text{clk}+\text{pos}, \text{addr}, \text{value})$ and receives the new one. In panel a the **LoadWord** row sits in the branch's delay slot, so its successor arrives through the state register as the branch target rather than $\text{pc}+4$. The rows compute $\text{op_a} \leftarrow \text{op_b} + \text{imm}$ with a byte-checked result and witnessed carries (**AddSubImm**), $\text{taken} \leftarrow [\text{op_b} \neq \text{op_c}]$ with $\text{next_next_pc} \leftarrow \text{taken} ? \text{target} : \text{next_pc}+4$ (**Branch**), and the aligned four-lane load of panel b (**LoadWord**).

Proof. Each access sends its previous tuple and receives its new tuple on the memory bus, and its constraints enforce that the new timestamp strictly exceeds the previous one. Balance of the memory bus, restricted to the tuples with address a , gives a graph as in Lemma 6.1 with the initial-state row as source and the final-state row as sink; strictly increasing timestamps exclude cycles, so the accesses form one path, and along the path a read carries its previous value forward unchanged. $\square$

Registers. The `MemoryBump` unit inserts, for every register at its first touch in a shard, a shadow read at $(\text{shard}, 0)$. Since `clk` restarts at zero in every shard and real accesses sit at positions 1..4 of a tick, the previous access of a register is always in the current shard. The register variant of the access columns therefore drops `prev_shard`, the compare selector and one limb, which is re-derived linearly from the others: 10 columns become 7 on every register access of every instruction.

Shard-local memory. Within a shard, the first access to a word must consume a tuple that no access in the shard produced, and the last access produces a tuple that nothing in the shard consumes. The `MemoryLocal` unit records, for every word touched in the shard, its initial and final $(\text{shard}, \text{clk}, \text{value})$; it sends the initial tuple on the memory bus (so the first access can consume it) and receives the final one, and it forwards both as *global* interactions (Section 6.4) so that they are matched across shards. On a `reth` block (the Ethereum execution-client workload of Section 11) a few hundred thousand words are touched per shard, most of them the hot working set touched again in the next shard; the two global rows per touched word are the largest single contribution to trace area, which is why the shard size matters (Section 11).

Whole-execution memory. `MemoryGlobalInit` carries the memory’s initial contents (the program image and the hint data written into untouched words) and `MemoryGlobalFinalize` its final contents. Both witness the word as 32 boolean columns, which is what makes every value that enters memory byte-shaped by construction; addresses are 32-bit decomposed and proved strictly increasing along the table through a running comparison against the previous address, with the address cursors published as public values so the recursion can check that consecutive tables tile the address space without overlap. The program image itself is bound by the verifying key: its digest (Section 6.4) is a public parameter, and the init table must consume it.

What is byte-checked. Every value that enters a register is byte-checked by the frame at the write, and every value a precompile writes to memory is byte-checked or bit-decomposed by the precompile unit; memory-instruction rows therefore omit the two additional byte lookups per word that accesses otherwise carry.

6.4 Cross-shard interactions: the global digest

Interactions whose two sides live in different shards cannot be balanced by a per-shard fraction sum. These include the initial and final tuples of every shard-touched word, memory initialisation and finalisation, and syscall pairings. ZIREN carries them in an elliptic-curve digest of the kind SP1 introduced [Suc24]. An interaction tuple $\mathbf{m} \in \mathbb{F}^7$ is mapped to a point of $E : y^2 = x^3 + 3\zeta x - 3$ over the degree-seven extension $\mathbb{F}_{p^7} = \mathbb{F}[\zeta]/(\zeta^7 + 2\zeta - 8)$; sends contribute the point and receives its negation, and the sum over the whole execution must be the identity. A shard proof publishes its running sum, and the block verifier adds them.

The `Global` unit has one row per such interaction. It witnesses the point, checks the curve equation, and fixes the sign of y by confining its top limb to one of two disjoint bands, one for sends and one for receives. The running sum is carried on an accumulation bus, row i receiving (i, S_i) and sending $(i + 1, S_{i+1})$ with the chord identities enforcing the addition, and the public-values table closes the chain.

The two exceptional cases of the chord law are excluded separately, and by different mechanisms. Both identities carry the factor $(x_2 - x_1)$:

$$C_x = (x_1 + x_2 + x_3)(x_2 - x_1)^2 - (y_2 - y_1)^2, \quad C_y = (y_1 + y_3)(x_2 - x_1) - (y_2 - y_1)(x_1 - x_3).$$

At $P_2 = -P_1$ we have $C_x = -4y_1^2$, which is non-zero whenever $y_1 \neq 0$, so the identity case is rejected by C_x itself. At $P_2 = P_1$ both coordinate differences vanish, so C_x and C_y hold for *every* P_3 and the identities alone

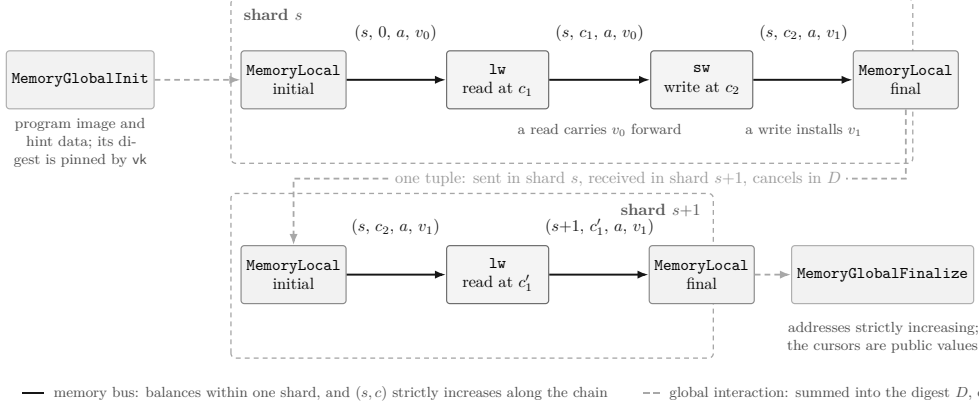


Figure 4: The chain of one address a across a shard boundary. A shard proof can balance only the tuples whose two sides it contains, so the **MemoryLocal** rows stand in for the accesses it cannot see: the initial row supplies the tuple the shard’s first access consumes, and the final row consumes the tuple its last access produces. Those two tuples are then the *same* tuple on either side of the boundary, carried with opposite signs into the digest D of Section 6.4, which is why the chain closes over the execution without any shard proof depending on another. The two ends belong to the whole-execution tables rather than to a shard.

would leave the next running sum unconstrained but for the curve equation. The unit therefore witnesses $(x_2 - x_1)^{-1}$ and constrains $(x_2 - x_1) \cdot (x_2 - x_1)^{-1} = 1$ on every real row, which makes $x_2 \neq x_1$ a constraint rather than an assumption; since both points are on the curve, $x_2 = x_1$ forces $y_2 \in \{y_1, -y_1\}$ and there is no third case. The first row is separately safe: the sum starts at a fixed public offset point D chosen with $\pm D$ outside the image of the map. What remains is a running sum at some later row coinciding with the next interaction’s point, which the inverse makes unprovable rather than unconstrained, and whose probability Assumption 6.4 bounds by the unquantified $\varepsilon_{\text{digest}}$.

One departure from SP1 matters. SP1 hashes the message to the curve with a permutation constrained inside the unit, so its points are random-oracle outputs. ZIREN applies no hash: the message *is* the x -coordinate, up to a prover-chosen byte **offset** in the low positions of x_6 that makes the curve’s right-hand side a square, so that a point with that x exists. That is what reduces the unit from a few hundred columns to a few dozen, and it changes the security argument rather than merely its constants.

6.5 Security of the digest

The lift is a map on *pairs*, not on messages: $\text{enc}(\mathbf{m}, \text{offset})$. The offset is a witness column, and the unit constrains only that it is a byte occupying the low positions of x_6 ; it does not constrain it to be canonical. The honest prover takes the smallest admissible offset. The implementation scans $\text{offset} = 0, \dots, 255$ and stops at the first value whose x makes $x^3 + 3\zeta x - 3$ a square in $\mathbb{F}_{p^7}$, so that the point exists. Minimality is not an AIR constraint, so a satisfying trace may carry any admissible offset. We therefore write $\mathcal{P}$ for the *reachable pairs* $(\mathbf{m}, \text{offset})$ that an interaction of a satisfying trace can carry, given the range facts each sending unit enforces and the byte bound on the offset, and reserve $\mathcal{M}$ for the projection of $\mathcal{P}$ onto messages. $\mathcal{M}$ is far smaller than $\mathbb{F}^7$, since every limb of a memory or syscall tuple is a byte, a bounded clock limb or a bounded address, and the whole argument rests on that; the offset multiplies the reachable set by at most 256.

Two consequences follow from the offset being free. First, cancellation is over pairs: a send and its matching receive contribute $\pm \text{enc}(\mathbf{m}, \text{offset})$ and cancel only if they agree on the offset. A trace whose two sides choose different admissible offsets for the same message does not balance and is rejected; the freedom costs completeness, not soundness, on that axis. Second, and this is the axis that matters below, the adversary searching for a vanishing combination searches over $\mathcal{P}$ rather than $\mathcal{M}$.

Lemma 6.3 (injectivity of the lift). *Assume the message limbs satisfy the range facts the sending units enforce, so that $256 m_6 + \text{offset} < p$. Then enc is injective: limbs $x_0, \dots, x_5$ are $m_0, \dots, m_5$; x_6 is m_6 shifted*

above a low byte holding the offset, which the bound makes separable; and the two sign bands are disjoint. So $\mathbf{m}$ and the offset are recoverable from the point.

Injectivity makes the digest a faithful encoding of the multiset of *pairs*, but it is not what makes the argument sound. If a claimed execution has send and receive multisets $S \neq R$ over $\mathcal{P}$ whose digests agree, then $\sum_i c_i \text{enc}(\mathbf{m}_i, \text{offset}_i) = O$ for the non-zero integer vector $c = \text{mult}_S - \text{mult}_R$. Equality of encoded multisets implies equality of message multisets only after projecting away the offsets, which is legitimate precisely because the lift is injective on pairs (Lemma 6.3). Soundness is therefore exactly:

Assumption 6.4 (no reachable vanishing combination). Let λ be the security parameter, let N_{int} bound the interactions a block proof may contain and B the per-interaction multiplicity bound, and let $\mathcal{P}$ be the reachable pairs above. For every adversary $\mathcal{A}$ running in time at most $T(\lambda)$ and making at most $q(\lambda)$ queries to the hash functions of the protocol,

$$\Pr [\mathcal{E}(\mathcal{A})] \leq \varepsilon_{\text{digest}}(\lambda, T, q, N_{\text{int}}, B),$$

where $\mathcal{E}(\mathcal{A})$ is the event that $\mathcal{A}$ outputs reachable pairs $\{(\mathbf{m}_i, \text{offset}_i)\}_{i \leq N_{\text{int}}} \subseteq \mathcal{P}$ together with a non-zero integer vector $c \in \mathbb{Z}^{N_{\text{int}}}$ satisfying $\|c\|_\infty \leq B$ and $\sum_i c_i \text{enc}(\mathbf{m}_i, \text{offset}_i) = O$ in $E(\mathbb{F}_{p^7})$.

The assumption is that $\varepsilon_{\text{digest}}$ is at most the level Section 8.4 targets, at the interaction and multiplicity bounds of a block proof; that is what Theorem 8.1 takes as a hypothesis, and this machine's parameters are fixed rather than a family, so the statement is concrete. We do not claim a value for it; the paragraph below explains why, and Section 8.4 carries it as a named open component rather than folding it into a bit total.

This is the weakest link in the accounting, and we state it rather than a bit figure for two reasons. Hashing to the curve, as SP1 does, would make the points random-oracle outputs and support a clean discrete-logarithm statement about points no adversary can steer. Here the prover chooses points from a structured set, and reads a message off any target x -coordinate by inverting the limb layout of Lemma 6.3; the uncanonicalised offset widens that set by a further factor of at most 256. The sparsity of $\mathcal{P}$ blocks the obvious attack of choosing a generator G , selecting scalars that sum to zero and reading their x -coordinates as messages, since almost no such x has the limb shape a reachable pair requires. This reasoning depends on every sending unit bounding every contributed limb, an arithmetisation obligation not collected in one certificate today. Sparsity also does not yield a bit estimate: the relevant generalised-birthday cost falls with the number of lists and must be bounded using the interactions a prover can place in a trace, not only $|E(\mathbb{F}_{p^7})| \approx 2^{217}$. That analysis remains open. Section 8.4 therefore lists the digest as a separate computational component.

6.6 Trace uniqueness

Theorem 6.5 (From unit determinism to trace uniqueness). *Fix a program image, public values and a valid shard. If every unit of the shard is deterministic in the sense of Definition 2.4, then the rows reachable from the entry state along the state and memory buses are determined by the program image and the public values: any two valid shards with the same program image and public values agree on those rows up to permutation.*

Proof. By Lemma 6.1 the instruction rows of either shard form one chain from the entry state; write $r_1, \dots, r_N$ for the first shard's chain. We show by induction on i that r_i has the same inputs and outputs in both shards. The inputs of r_i are its received state tuple s_{i-1} , its fetched instruction, and the previous values of the addresses it accesses. s_0 is public and s_{i-1} is the output of r_{i-1} by induction. The fetched instruction is a function of $\text{pc}(s_{i-1})$ because the program bus is received only by the preprocessed program table, which is committed. The previous value of an address is, by Lemma 6.2, the value written by the last earlier access to that address, or the initial value from the memory-initialisation table, which is bound to the program image and the public address cursors; earlier accesses belong to rows r_j , $j < i$, whose outputs agree by induction. So the inputs of r_i agree, and determinism of its unit gives that its outputs agree. Rows of non-instruction units (the memory-initialisation and finalisation tables, the shadow reads, the digest rows, the tables) are determined in the same way from the bus tuples they receive, which are outputs of instruction rows or public. Rows that participate in no bus reachable from the entry state are not covered by this argument; the arithmetisation admits none, because every real row carries a frame or a memory access, but that is a property of the unit set rather than of the buses, and we do not prove it here. $\square$

The theorem is stated, not machine-checked; its premises are the unit-level determinism statements, of which 17 of 106 are machine-checked today and the rest are assumed (Section 10.2), and the two bus lemmas are the standard arguments of the multiset construction.

6.7 Preprocessed tables

The **Byte** table has 2^{16} rows indexed by a byte pair and one multiplicity column per operation (bitwise operations, shifts, comparisons and 8/16-bit range checks); every carry, borrow, comparison and range check in the machine is an interaction with it. The **Range** table serves the clock limbs, and the **Program** table holds the decoded program image. Preprocessed tables are committed at setup as part of the verifying key and opened at the same points as the main trace.

6.8 The shard-level lookup argument

This is the bus argument of §5.2.

One **LogUp-GKR** instance proves all interactions of a shard on the program, state, memory, byte, range, syscall, global and accumulation buses. After committing the trace, the verifier samples $\alpha, \beta \in \mathbb{F}_{p^4}$. Every interaction (**bus**, $\mathbf{f}$, m) contributes $m/(\alpha - \text{bus} - \sum_{j \geq 1} \beta^j f_j)$, and the prover shows that sends and receives have equal sums. These fractions are leaves of a layered GKR circuit. Sumcheck proves its transitions from the root down, and the final layer produces multilinear evaluation claims on the participating columns for the polynomial commitment. A reth shard has on the order of 10^8 leaves. Section 11 shows that this circuit is the largest GPU-kernel consumer; its leaf count is the number of (row, interaction) pairs, so interactions per row are a separate cost from columns.

Soundness of the fraction identity is $\deg / |\mathbb{F}_{p^4}|$ per sumcheck round plus the collision probability of the fingerprint, which grows with the number of leaves; Section 8.4 lists it as one component of the accounting. The checked-in schedule grinds the **LogUp-GKR** challenge by 16 bits (Table 3); the performance baseline of Section 11.4 was measured with none (§8.4).

6.9 The shard-level zerocheck

This is the constraint argument of §5.2.

The polynomial constraints of all units are proved by *one* zerocheck. The verifier samples τ for a random linear combination across units and μ across the constraints of a unit; for each unit the polynomial $C_c(\mathbf{X}) = \sum_k \mu^k c_{c,k}(\text{row}(\mathbf{X}))$ must vanish on the unit’s real rows. The prover proves $\sum_{\mathbf{b}} \text{eq}(\mathbf{b}, \mathbf{r}) \sum_c \tau^c C_c(\mathbf{b}) = 0$ for a random $\mathbf{r}$ by sumcheck over the tallest unit’s variables, with shorter units handled by an analytic “virtual $\geq$ ” factor rather than padding, and the final evaluation claims on the units’ columns are again handed to the commitment. The constraint degree is three, which fixes the degree of the round polynomials at four; the constraint-evaluation kernel on the GPU is compiled per unit from the same constraint description that the verifier uses, except for the widest accelerator units, which are interpreted (Table 5), and a mismatch between the two is caught by host verification (Section 11).

The zerocheck’s claims are seeded from the **LogUp-GKR** openings so that the two protocols share one opening of each column: after the two sumchecks, what remains is a set of (unit, column, point, value) claims, of the order of $3.6 \cdot 10^4$ per shard, and turning those into a single claim on a single committed polynomial is the job of the commitment layer.

7 The jagged commitment

After the lookup argument and zerocheck, the verifier holds evaluation claims on individual unit columns. A shard of the **reth** workload has about $3.6 \cdot 10^4$ columns whose heights range from 32 rows to millions. Committing each column or unit separately would cost one Merkle root and query phase per commitment, and recursion would depend on the unit set of each shard. **ZIREN** instead reduces all claims to one evaluation of a *logical* dense polynomial and opens it once. The data may reside under several ordered input commitments, with the preprocessed round first and main round last, but one jagged reduction, one branching-program assist and one batched PCS opening cover all roots.

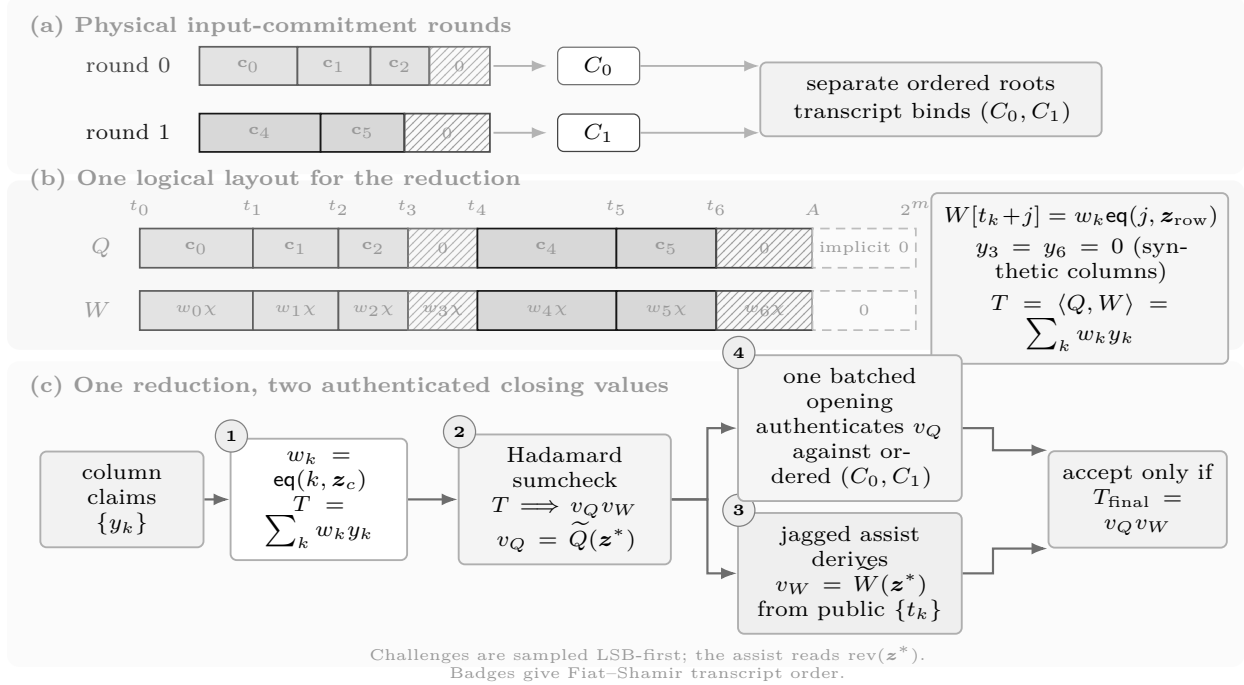


Figure 5: The multi-round jagged opening. Physical input rounds retain separate ordered roots (a), while the reduction aligns the logical dense table Q with its public-boundary-derived weight table W (b). The Hadamard sumcheck closes with $v_Q = \tilde{Q}(z^*)$ and $v_W = \tilde{W}(z^*)$; the jagged assist derives v_W , and one batched WHIR/BaseFold opening authenticates v_Q against every ordered root (c).

The property this buys is modularity in the sense of Section 4: because the verifier of the reduction depends only on the logarithm of the committed area and on a fixed description of the column geometry, a unit can be widened, narrowed, added or removed without redesigning the layer above it. The block's internals can be revised, as they were four times in Section 11.4, while the reduction protocol remains the same.

7.1 Input commitments, dense packing and stacking

Figure 5 shows the two views this section connects: the ordered input rounds, each keeping its own root, and the single logical dense table that the reduction opens.

Call a preprocessed or main commitment an *input-commitment round*, to distinguish it from a folding round of WHIR. Let there be R such rounds, in transcript order. Round r contains columns $\mathbf{c}_{r,0}, \dots, \mathbf{c}_{r,K_r-1}$ of heights $h_{r,0}, \dots, h_{r,K_r-1}$. Its real area is

$$a_r = \sum_{k=0}^{K_r-1} h_{r,k},$$

and its commitment covers an area $A_r \geq a_r$ that is a multiple of the stacking height $H = 2^\ell$. The committed dense vector for the round is

$$\mathbf{q}_r = \mathbf{c}_{r,0} \parallel \dots \parallel \mathbf{c}_{r,K_r-1} \parallel \mathbf{0}^{A_r - a_r}.$$

The padding is part of what the root commits to; it is not the final power-of-two padding of a multilinear polynomial.

The reduction sees the rounds as the one ordered logical vector

$$\mathbf{Q} = \mathbf{q}_0 \parallel \mathbf{q}_1 \parallel \dots \parallel \mathbf{q}_{R-1}, \quad A = \sum_{r=0}^{R-1} A_r.$$

It pads $\mathbf{Q}$ with a final zero suffix to $M = 2^m$, where $m = \lceil \log_2 A \rceil$, and writes $\tilde{Q}$ for the resulting multilinear polynomial. The roots $C_0, \dots, C_{R-1}$ remain separate: logical concatenation does not replace them by a new root, and their order is load-bearing.

To make the column prefix sums describe every committed cell, the gap $A_r - a_r$ is represented by explicit synthetic zero columns. Each such column has zero claim and height at most the common row cube 2^n . Under a recursion area pin the gap is split into a fixed number of columns, allowing zero-height columns, so the recursion program's column count does not depend on the actual row counts of a child. Let the resulting global columns, including these padding columns, have heights $h_0, \dots, h_{K-1}$ and prefix sums

$$t_0 = 0, \quad t_{k+1} = t_k + h_k, \quad t_K = A.$$

Thus a round's stacking padding appears at its true location in the middle of $\mathbf{Q}$; only the suffix from A to M is implicit.

Each round is cut independently into $S_r = A_r/H$ stripes of H consecutive entries and committed under its own root. Across all rounds there are

$$S = \sum_r S_r, \quad b = \lceil \log_2 \max(S, 1) \rceil$$

stripe positions in the batched opening, zero-padded to 2^b . Consequently the opening domain has $\ell + b$ variables and effective area $H2^b$. For a nonempty ordinary instance this is the same M used by the reduction. We write its point as $(\mathbf{z}_{\text{stack}}, \mathbf{z}_{\text{batch}})$, with the first ℓ coordinates selecting a position inside a stripe and the remaining b coordinates interpolating the ordered list of stripes across all input commitments.

7.2 The jagged reduction

Let $\mathbf{z}_{\text{row}} \in \mathbb{F}_{p^4}^n$ be the common row point left by the zerocheck, and let y_k be the claimed evaluation of global column k at that point, including the high-zero embedding needed when the column is shorter than the common row cube:

$$y_k = \sum_{j=0}^{h_k-1} Q[t_k + j] \text{eq}(j, \mathbf{z}_{\text{row}}).$$

The verifier samples a column point $\mathbf{z}_c \in \mathbb{F}_{p^4}^\kappa$, $\kappa = \lceil \log_2 K \rceil$, and uses the multilinear Lagrange weights

$$w_k = \text{eq}(k, \mathbf{z}_c), \quad T = \sum_{k=0}^{K-1} w_k y_k.$$

This is the batching used by the implementation: the weights are $\text{eq}(k, \mathbf{z}_c)$, not powers γ^k of a scalar challenge.

Define a length- M weight table W by

$$W[t_k + j] = w_k \text{eq}(j, \mathbf{z}_{\text{row}}) \quad (0 \leq j < h_k), \quad W[i] = 0 \quad \text{elsewhere.}$$

The definition and the column claims give the central identity

$$T = \sum_k w_k \sum_{j < h_k} Q[t_k + j] \text{eq}(j, \mathbf{z}_{\text{row}}) = \sum_{i \in \{0,1\}^m} \tilde{Q}(i) \tilde{W}(i). \quad (1)$$

The prover runs sumcheck on the Hadamard product $\tilde{Q}\tilde{W}$. Because both factors are multilinear, each round polynomial has degree at most two and is represented by its values at $0, 1, 2$. If the sumcheck challenges in sampling order are

$$\mathbf{z}^* = (r_0, \dots, r_{m-1}),$$

then the closing claim is

$$T_{\text{final}} = \tilde{Q}(\mathbf{z}^*) \tilde{W}(\mathbf{z}^*). \quad (2)$$

The proof carries $v_Q = \tilde{Q}(\mathbf{z}^*)$; the verifier derives $\tilde{W}(\mathbf{z}^*)$ from the public prefix sums and the transcript challenges. The implementation binds the stride-one, least-significant-bit variable first and stores $\mathbf{z}^*$ in sampling order. The branching program below reads an integer index most-significant bit first, so its trace point is $\text{rev}(\mathbf{z}^*)$.

7.3 The branching-program assist

Materialising W would cost $\Theta(M)$ extension-field elements. Instead, the jagged indicator is evaluated from the interval boundaries (t_k, t_{k+1}) . Proposition 3.2 of [HJR⁺25] expresses one interval’s contribution through a width-four read-once branching program [Bar89], whose multilinear extension is evaluated by the dynamic program of Holmgren–Rothblum [HR18]. Let

$$u_k = \text{bits}_{\text{BE}}(t_k) \parallel \text{bits}_{\text{BE}}(t_{k+1}),$$

using $m + 1$ bits per boundary. The assist proves the claim

$$\widetilde{W}(\mathbf{z}^*) = \sum_{x,y \in \{0,1\}^{m+1}} \sum_{k=0}^{K-1} w_k \text{eq}((x,y), u_k) \text{BP}(\mathbf{z}_{\text{row}}, \text{rev}(\mathbf{z}^*), x, y) \quad (3)$$

with one degree-two sumcheck over $2(m + 1)$ variables. At its closing point the recursion verifier evaluates one branching-program instance and the equality factors for the public prefix sums. Thus Equation (2) is tied to the same irregular layout that Equation (1) used, without allocating the dense weight table. The native verifier computes the same value by the closed form and replays the assist transcript; the recursion circuit verifies the assist’s closing identity explicitly. Equation (3) is the value claimed by that transcript and checked at its closing point.

7.4 One batched opening across all roots

The evaluation point for the PCS must span the full stacking domain. If the reduction point is shorter, the prover appends Fiat–Shamir coordinates after the assist until it has $\ell + b$ coordinates. Coordinates that extend a sub-stripe polynomial through a zero high half contribute the usual factor $\prod_j (1 - r_j)$ to the evaluation claim. In the ordinary multi-round layout all round padding is already explicit and $m = \ell + b$, so this extension is a no-op.

For round r and stripe s , let

$$e_{r,s} = \widetilde{\mathbf{q}_{r,s}}(\mathbf{z}_{\text{stack}})$$

be the stripe evaluation. The prover reports these evaluations in round-major, stripe-major order. The verifier zero-pads their flat list to 2^b and checks

$$v_Q = (e_{0,0}, \dots, e_{0,S_0-1}, \widetilde{e_{1,0}, \dots, e_{R-1, S_{R-1}-1}})(\mathbf{z}_{\text{batch}}). \quad (4)$$

One stacked WHIR or BaseFold proof then authenticates every $e_{r,s}$ against the ordered roots $C_0, \dots, C_{R-1}$ at the common stack point. Commitments therefore remain per input round, while the expensive reduction, assist and PCS opening are each performed once. All input rounds must use the same stacking height and the same PCS mode; mixing WHIR data and BaseFold data would give the verifier a different transcript and is rejected.

The Fiat–Shamir order is part of the protocol. After the commitments and the column claims have been absorbed by the surrounding shard protocol, the prover and verifier perform, in order:

1. sample $\mathbf{z}_c$ and form the column-claim mixture T ;
2. run the m -round Hadamard-product sumcheck, obtaining $\mathbf{z}^*$ and v_Q ;
3. run the $2(m + 1)$ -round branching-program assist at $\text{rev}(\mathbf{z}^*)$;
4. sample any point-extension coordinates; and
5. run one batched stacked WHIR or BaseFold opening against all input roots.

Changing this order, the round order, or the reversal convention changes the challenges and is therefore a protocol change.

Soundness. The column-point batching, the m -variable product sumcheck and the $2(m+1)$ -variable assist are separate algebraic error terms. They are included, together with the PCS terms, in the round-by-round accounting of Section 8.4. For the checked-in schedule, each algebraic term is negligible over $|\mathbb{F}_{p^4}| \approx 2^{124}$; the separate lookup-grinding mismatch and digest assumption are stated there.

7.5 The committed geometry, and why it is quantised

Two constants fix the layout. The stacking height is $H = 2^{21}$, so a round is committed as a whole number of 2^{21} -cell blocks; and a column's height is at most the row cube 2^{22} , which bounds how much of a gap one synthetic column can absorb.

A round's committed area is not its real area rounded up to the next block. It is placed in a *bucket*: with $B = \lceil a_r/H \rceil$ blocks of real data, the commitment covers $\hat{B}$ blocks, where $\hat{B} = B$ while $B \leq 4$ and otherwise $\hat{B}$ is the next multiple of eight, giving the bucket sequence

$$1, 2, 3, 4, 8, 16, 24, \dots \text{ blocks.}$$

The padding columns that represent the resulting gap are counted from the bucket $\hat{B}$, not from the real count B , which is what makes their number a function of the bucket alone. For a round whose area is not pinned, which is every core round, the gap is split into

$$\left\lceil \frac{(\hat{B} - \hat{B}_{\text{prev}})H}{2^{22}} \right\rceil \text{ columns,} \quad \hat{B}_{\text{prev}} = \text{the bucket below } \hat{B},$$

at least one, which is the number the *widest* gap of that bucket would need. A narrower gap simply leaves the trailing columns short or empty, which the verifier already accepts. For a round whose area is pinned, as the compress machine's is, the split is a fixed column count whatever the gap.

Both rules exist for the same reason, and it is not a proving cost. They make a shard's committed geometry, and therefore the recursion program that verifies the shard and that program's verifying key, a function of the shard's chip set and its two committed-block buckets rather than of its exact row counts. Round the area up to the next block instead, or let the padding-column count follow the gap, and each distinct row profile becomes a distinct verifier program with a distinct key. Under the quantised rule the key space is finite and small enough to enumerate offline, one representative per (chip set, preprocessed bucket, main bucket) class, which is what lets a verifier hold a fixed allowlist of recursion keys that covers every shard shape the machine's declared clusters admit (Section 9). The price is the padding inside a bucket, which the prover commits to and opens like any other cell.

Three things are never materialised, which is what keeps the reduction's memory cost proportional to the real data rather than to M . The weight table W is evaluated from the interval boundaries by the assist of §7.3. The zero suffix from A to M is implicit: the sumcheck treats the tail as zero rather than allocating it, which is why the inter-round padding has to be explicit while this suffix need not be. And the stripe evaluations of Equation (4) are read off the same interleaved layout the round was committed from, so a round is traversed once for its commitment and once for its opening.

7.6 Instantiation on the inner and outer rings

The same jagged layer is instantiated twice. On the inner ring, core and compress shards use the stacked WHIR opening of Section 8; on the outer *wrap* ring, the final recursion shard uses the FRI-based BaseFold-style instantiation described in Section 9. Dense packing, explicit inter-round padding, the Hadamard-product reduction, the branching-program assist and the transcript order are shared; only the proof that authenticates the stripe evaluations in Equation (4) differs.

8 The opening, and what it costs in soundness

WHIR [ACFY24b] is the polynomial commitment of §5.2. The evaluation claim $v_Q = \tilde{Q}(z^*)$ that the jagged reduction of Section 7 leaves is authenticated with it, an interactive oracle proof of proximity (IOPP) for *constrained* Reed–Solomon codes, that is, for codewords whose multilinear also satisfies a stated weighted sum.

Table 3: Checked-in Jagged-WHIR schedule. OOD entries count samples after committed folds; no folding challenge is ground. The stripe bound covers both ordered input-commitment rounds.

Profile	folds	queries	OOD	query grind	batch grind	LogUp-GKR grind	stripe bound	final variables
core	[3, 6, 6]	[124, 88, 85]	[2, 2]	22 bits	14 bits	22 bits	256	6
compress	[3, 6, 6]	[124, 88, 85]	[2, 2]	22 bits	14 bits	22 bits	64	6

We recall the protocol in the form it is implemented, then give the commitment layout and the soundness accounting. This layer holds the block’s principal configuration knob (§E.2): at a fixed soundness regime the schedule trades proof size against prover time, and an integrator who wants a smaller proof pays for it here. The accounting below, and the report it generates for a given configuration, are part of the collateral of §E.4.

8.1 The protocol

WHIR tests membership of a function $f : L \rightarrow \mathbb{F}$ in $\text{CRS}[\mathbb{F}, L, m, \hat{w}, \sigma]$ (Section 2). One iteration with folding factor k runs k sumcheck rounds, sends a folded oracle re-encoded at the round’s rate, answers out-of-domain samples, answers t shift queries by opening 2^k symbols each, and recurses on a new constrained code with k fewer variables; after the last iteration the final polynomial is sent in the clear. The out-of-domain samples are what distinguish WHIR from BaseFold: they pin the folded oracle to a unique nearby codeword before the shift queries are drawn, so the per-query soundness contribution is governed by the proximity parameter of the regime, which at a given rate is what the out-of-domain step pins down. The first prover of this generation used BaseFold at a fixed rate with a full query set in every round; WHIR at the same provable level uses fewer queries in its later rounds and a handful of committed oracle rounds rather than one per variable, and shares the prover-side skeleton (encode, Merkle-commit, fold), so the switch required no new kernels beyond the out-of-domain evaluation and the monomial-basis weights.

8.2 Commitment layout

ZIREN uses the *interleaved* commitment: a polynomial in ν variables is reshaped into a $2^{\nu-k_0} \times 2^{k_0}$ matrix whose column c is the stride- 2^{k_0} slice $f[h \cdot 2^{k_0} + c]$, each column is Reed-Solomon encoded independently (coefficients zero-padded, one DFT per column), and, for a single stacked stripe, Merkle leaf j holds the 2^{k_0} column values at domain point x_j . Folding a leaf by the round’s challenge $\mathbf{r}$ gives the monomial evaluation of the folded polynomial at x_j , so a query answer enters the next sumcheck as an ordinary constraint whose weight folds in closed form.

This layout is applied separately to every input-commitment round of Section 7.1; those roots are not merged. The batched opening evaluates every stripe at the common stack point, flattens the results in input-round order, and checks their interpolation at the batch point against v_Q before authenticating them against all roots. Within an input root the stripes are grouped 32 to a leaf, so a leaf of the first WHIR oracle authenticates one coset row from each stripe, 32×2^{k_0} field elements. The folding factor k_0 of the first WHIR round is therefore chosen small, because the leaf width multiplies every query of that round and re-hashing those leaves is the recursion verifier’s dominant cost; later WHIR rounds query a single folded polynomial and fold more variables at a time.

Each committed WHIR round re-encodes the folded polynomial with one DFT per column at the new rate, and the opening of a shard of the **reth** workload costs about 39 ms of host-visible time per shard on the GPU with device-resident rounds; the round-0 re-encoding of the stripes is the largest single term. No proof-of-work is spent on the folding challenges; all grinding sits on the query phases, where a bit of work buys a bit of security against the query-sampling adversary.

Table 3 gives the concrete schedule used by the checked-in core and compress profiles. The first oracle has rate 2^{-2} ; each recommitment lowers the rate by three further powers of two. At log stacking height 21, the three folds consume 15 variables and the remaining six-variable polynomial is sent in the clear after its final query phase.

8.3 The outer ring: a BaseFold instantiation and what its verifier owes

The wrap proof of Section 9 does not use WHIR. It commits its dense polynomial with a FRI-style BaseFold instantiation over the same field but with a 254-bit hash, folding one variable per round: each round emits one univariate sumcheck message for the current variable, samples a challenge that is shared between the sumcheck and the fold, and opens the query chain at the folded oracle, until a short final polynomial is sent in the clear. The shared challenge is what ties the two structures together: the sumcheck certifies the claimed value while the query chain certifies that the folded oracles are consistent with the committed one.

Because the two are separate structures, a verifier that checks only one of them is not checking the protocol, and this is where the outer ring differs from the inner one in how much its verifier had to be repaired. The recursive BaseFold verifier is the last one in the chain, the artefact a Groth16 circuit consumes, and it originally asserted only the query chain: it folded the openings, compared them round by round, and never evaluated the sumcheck’s terminal identity, so nothing forced the claimed evaluation to be the value the committed polynomial takes at the point. It also took the number of queries from the proof, as the smaller of the scheduled count and the length of the list the prover supplied, which lets a prover shorten its own query phase. Both are fixed: the terminal identity is asserted in-circuit, and the query count comes from the schedule with a truncated list rejected. Two rules generalise from them, and they apply to any recursive verifier of a folding argument: every algebraic identity the native verifier checks must appear in the circuit, because the circuit is the only verifier an on-chain consumer runs; and no quantity that a soundness bound depends on may be read from the proof.

The wrap circuit’s public interface is equally load-bearing. It takes three public inputs: the guest program’s verifying-key hash, the digest of the guest’s committed values, and the root of the recursion-key allowlist. Its verifying key is fixed in-circuit, so a proof cannot be transplanted onto a different recursion stack. Section 9 describes why the allowlist root has to leave the tree as a public input rather than remain a witness, and Table 12 records the release in which it did not.

8.4 Soundness accounting

Round-by-round soundness of the *interactive* protocol is bounded by the sum of the errors of its rounds. The Fiat–Shamir compilation then costs the oracle-query factor of Theorem 8.1, so every figure in this section is a bound on the interactive protocol and not a security level for the compiled one. The grinding bits of Table 3 are counted inside the per-round terms below, not as a separate discount against that factor. Each term is expressed as bits = $-\log_2$ of its error, computed with the `soundcalc` tool [Eth26a] on the configuration file kept with the source, and then the union bound over the terms; the level of a proof is the latter, not the smallest term. Other projects publish calculators of their own, RISC Zero’s being executable for its own parameters [RIS24]; the numbers below are not comparable to theirs unless the same components are charged.

Regimes. In the *unique-decoding* regime (UDR) the proximity parameter is $\delta = (1 - \rho)/2$ and a shift query contributes $-\log_2(1 - \delta) = -\log_2 \frac{1+\rho}{2}$ bits: at a rate of 1/4 this is 0.678 bits and it approaches one bit as the rate falls; no conjecture is needed. In the *Johnson* regime $\delta = 1 - \sqrt{\rho} - \eta$ and a query is worth up to $\log_2(1/\sqrt{\rho})$ bits, which the correlated-agreement results of [BSCI⁺20, ACFY24b] give up to the Johnson bound without conjecture; only the further reach towards capacity, $\delta \rightarrow 1 - \rho$, is conjectural, but folding terms $\approx (2^m 2^k / \rho) / |\mathbb{F}_{p^4}|$ that do not improve with more queries cap what that regime can add over a degree-four extension of a 31-bit field. ZIREN therefore sets its query counts in the unique-decoding regime so that every committed WHIR round clears the same per-round target; with t queries and b bits of query grinding a round is worth $t \cdot (-\log_2 \frac{1+\rho}{2}) + b$ bits.

Components and their composition. The transcript’s components are the out-of-domain samples and shift queries of each round, batching of the column claims, the final polynomial, each folding step, the reduction to the dense polynomial (Section 7), the zerocheck, the LogUp-GKR argument and the hash. `soundcalc` evaluates each component as a function of trace length, column count, schedule and field. The shard proof’s error is the sum of the component errors, so its level in bits is $-\log_2$ of that sum. One further term sits outside `soundcalc`’s model: the structured cross-shard digest assumption of Section 6.4. Modelling

Table 4: Component errors of the checked-in configuration, in bits ($-\log_2$ of the error), grouped from the 24, 24 and 26 components the calculator reports for the three profiles [Eth26a]. A group of n components is the union over them, which is why a group can be worth less than its smallest member: the fifteen folding steps of a core proof are each at least 103 bits and together 100.93. The last row is the union over all components of that profile, and it is what §8.4 reports; it charges one component the calculator does not carry, the final query phase, which the tool folds into the last committed round and which is worth $85 \cdot 0.9993 + 22 = 106.9$ bits at the rate that round commits (2^{-11}), costing the core union 0.04 bits; it is a bound on the interactive protocol, before the oracle-query factor of Theorem 8.1 and before the digest term of Assumption 6.4. The wrap profile folds one variable per committed round and draws no out-of-domain sample, so two rows do not apply to it.

Component group	core	compress	wrap
Query phases	105.00	105.00	104.00
Folding steps	100.93	100.93	—
Committed rounds	—	—	101.00
Final polynomial	106.00	106.00	—
LogUp-GKR	106.00	118.00	118.00
Batching	107.00	110.00	122.00
Zerocheck	109.00	114.00	115.00
Reduction to the dense polynomial	116.00	116.00	116.00
Out-of-domain samples	213.00	213.00	—
Union over the profile	100.70	100.76	100.83

the Poseidon2 transcript as a random oracle is the setting in which the compiled statement holds rather than an additive error term.

Table 4 is that accounting: every component the tool reports, grouped, for each of the three profiles.

The schedule is solved rather than listed. Each committed round’s query count is the least one that reaches a per-component target of 106 bits at that round’s rate net of its grind, which is what puts the query phases of Table 4 at 105 bits for a pair of rounds and leaves the union over a profile above 100. The generated report labels core and compress at 103 bits and wrap at 102 because that report’s **total** field is the *minimum component level*, not the union bound across components. What binds is no longer a term grinding can move: for core and compress it is the fifteen folding steps, whose error $\approx 2^m 2^k / \rho |\mathbb{F}_{p^4}|$ is set by the field and the schedule and does not improve with queries, and for the wrap profile, which folds one variable per round with no committed WHIR round (§8.3), its twenty-one committed rounds. Summing each profile’s own components gives 100.70 bits for core, 100.76 for compress and 100.83 for wrap, and 99.18 bits for the union over the three. That last figure counts each profile once. A block proof contains many nodes of each kind, and Theorem 8.1 charges every one of them: at the shard and node counts of the block measured in Section 11, roughly sixty core proofs, the sixty leaf proofs above them and thirty compose proofs under one wrap, the same components give 93.50 bits. Even that is a within-tree subtotal, before the adversary’s Q random-oracle queries and the separate digest term.

From the interactive bound to a compiled level. The step the accounting owes is the one the previous paragraph stops at. By Theorem 8.1 the compiled argument’s error is at most $Q \cdot \sum_i \varepsilon_i + \varepsilon_{\text{digest}}$ against an adversary making Q queries to the oracle, so the level in bits is the interactive figure minus $\log_2 Q$, with the digest term still outstanding:

$$\lambda(Q) = 93.50 - \log_2 Q \quad (\text{node-weighted}), \quad 99.18 - \log_2 Q \quad (\text{per profile}).$$

The grinding is already inside those figures and does not discount Q a second time: b bits of query grinding cost an adversary 2^b oracle calls per attempt, which is exactly the factor by which the per-round error was reduced. At $Q = 2^{40}$ the node-weighted bound is 53.5 bits, at $Q = 2^{60}$ it is 33.5, and at $Q = 2^{80}$ it is 13.5.

We state this rather than leave it implicit because it is the honest reading of our own theorem, and it is not a three-digit number. Reaching a level λ against a budget Q needs an interactive bound of $\lambda + \log_2 Q$, which this schedule does not provide and which grinding cannot buy at the rate required: a bit of security bought this way costs the honest prover a factor of two as well. What the configuration does establish is the

interactive bound of Table 4, and what it does not establish is a compiled level against an unbounded-query adversary. Closing that gap needs either a tighter compilation analysis for this protocol class, where the factor Q is a worst case rather than a tight bound, or a schedule solved for the compiled target rather than the interactive one. Neither is work this paper has done. The performance baseline in Section 11.4 used zero lookup grinding. The checked-in report therefore does not assign a numerical level to that measured baseline, and this paper does not transfer the corrected schedule’s values to the zero-grinding measurements. A release claim requires regenerating the report from the exact runtime configuration and instantiating the remaining terms.

Theorem 8.1 (Conditional composition). *Work in the random-oracle model, and let an adversary make at most Q oracle queries. Let every proof in the tree, including each shard, leaf and compose proof, be the Fiat–Shamir compilation [BSCS16] of an interactive protocol that is round-by-round knowledge sound with error $\varepsilon_i = \sum_c \varepsilon_{i,c}$. Let a block proof contain N such proofs, each verified once by its parent, with the root checked by the external verifier and every node’s verifying key bound by the allowlist of Section 9. Assume further that each node extractor succeeds on transcripts produced by its parent extractor, not only on transcripts drawn from an honest interaction. Then the block proof has knowledge error at most*

$$Q \sum_{i=1}^N \varepsilon_i + \varepsilon_{\text{digest}},$$

where $\varepsilon_{\text{digest}}$ is the error of Assumption 6.4.

Proof sketch. Compiling a round-by-round knowledge sound protocol with Fiat–Shamir costs a factor in the number of oracle queries: an adversary may rewind and retry, and the standard analysis charges Q attempts against the per-round error [BSCS16, CY24]. That accounts for the factor Q and for the fact that the random-oracle model is where the statement lives rather than an additive term in it. Within the tree, the extractor of the compiled argument runs the extractor of each verified proof in turn: the extractor for a compose node yields the transcripts of its children, and so on down to the shard proofs, whose extractor yields the witness rows. The allowlist hypothesis is what makes each step an extraction from the intended verifier; without it a node could verify a program other than the compose or leaf program. The hypothesis about parent-produced transcripts is the step this sketch does not discharge, and it is not free: a child’s extractor is invoked on a transcript that the parent’s extractor constructed, which is not the distribution against which its guarantee is stated. Given it, the failure events combine by the union bound. $\square$

For the block of Section 11, N is its core shards plus the leaf and compose nodes above them, of the order of two hundred proofs. We state the bound because it is what the union bound gives; published levels for other systems are per proof and do not compose over the tree.

Proof size. The shard proofs of the measured block total on the order of 10^8 bytes before recursion (§11.4); the published object is the compressed proof at the root. An instrumented pre-correction run recorded 617 618 bytes (603 KiB), of which 79% was the first WHIR oracle’s openings, 9.5% the later WHIR rounds and 6% the LogUp-GKR layer proofs. To first order, the size is $t_0 \cdot (\text{stripes}) \cdot 2^{k_0}$. The retained repository census is for an earlier configuration and does not reconstruct this exact serialized proof, so the value is a reported measurement rather than a result independently reproducible from the current artifact. Figure 6 records its instrumented breakdown.

9 Recursion

This section describes the block’s outward interface: what leaves it is the root of the tree built here, the compressed proof of Table 11, and what binds that proof to the advertised machine is the verifying-key allowlist of the last paragraph. Shard proofs are aggregated by a recursion machine: a small register VM over the same field whose programs are compiled from a description of the verifier, executed to a trace, and proved by the same unit/LogUp-GKR/zerocheck/Jagged/WHIR pipeline. Its units are the arithmetic of $\mathbb{F}$ and $\mathbb{F}_{p^4}$, memory, the Poseidon2 permutation, a select gate and a batched folding gate for the FRI-style query checks.

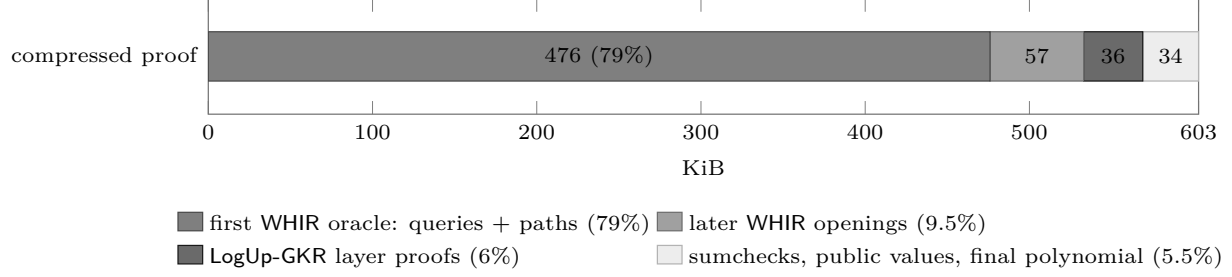


Figure 6: In the instrumented pre-correction proof (617 618 bytes), openings of the first WHIR oracle account for 79% because each query authenticates one coset row of every stripe. The retained census describes an earlier configuration and does not reproduce this exact file.

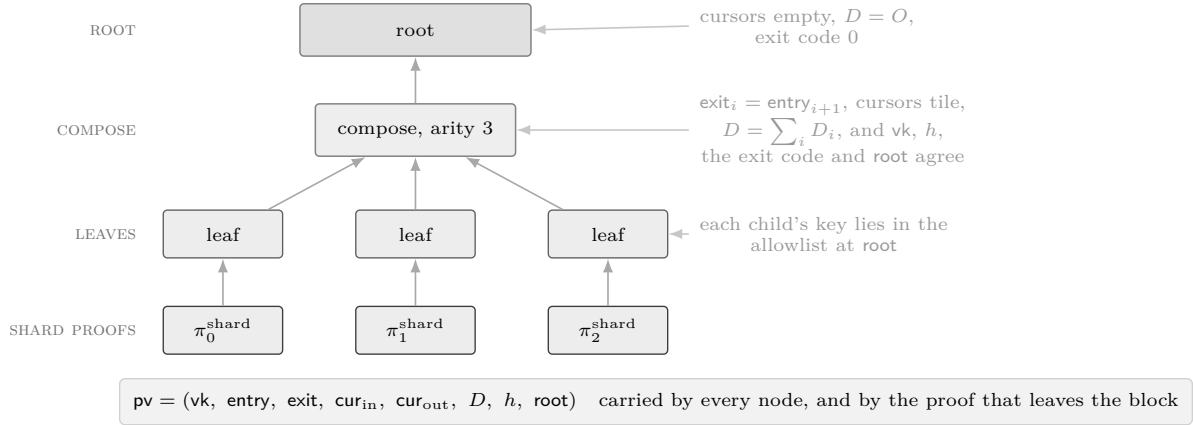


Figure 7: Every node of the tree carries the same public-value tuple, so a compose node checks its children by comparing tuples: the ranges must be adjacent, the cursors must tile and the digests must accumulate, while the verifying key, the output digest, the exit code and the allowlist root must agree. The root is where the memory argument and the cross-shard digest close, and where the exit code is required to be the one the relation admits.

Leaves. A *leaf* program verifies one core shard proof in four phases mirroring the host verifier: the transcript prologue (public values, commitment, unit dimensions), the LogUp-GKR layer replay, the zerocheck replay including the constraint evaluation of every unit present, and the jagged opening with its branching-program closing identity followed by the WHIR verifier. Because the jagged verifier depends only on the shard’s log-area and the constraint evaluation only on the unit set, the number of distinct leaf programs is small; on the single-GPU deployment consecutive shards of the same class reuse a device-resident proving key. Adjacent shards’ leaves are *fused* into one node where the shard shapes allow, and leaves are proved on the same GPU concurrently with core shards, so that the GPU’s time is shared between the two stages rather than serialised.

Figure 7 shows the tree, what each node checks on its children and what the root closes.

The compose relation. Write a node’s public values as

$$pv = (vk, entry, exit, cur_{in}, cur_{out}, D, h, code, root),$$

the verifying key of the guest, the entry and exit machine states, the memory-address cursors the node opens and closes with, the accumulated cross-shard digest, the digest of committed output, the exit code, and the root of the allowlist of admissible recursion keys. For a machine with a delay slot the entry and exit states are *pairs* (pc, pc_{next}) , because a row’s successor is data; adjacency therefore has to pin both components, and a defect in which only the first was pinned across shards is recorded in Table 12. A compose node of arity k proves the relation $\mathcal{R}_{comp}(pv; \pi_0, \dots, \pi_{k-1})$, which holds when each π_i verifies against a key that the allowlist at *root* contains, the children agree on vk, h and *code*, their $root_i$ all equal the root that membership is checked against and that the node publishes, their ranges are adjacent in the sense that $exit_i = entry_{i+1}$ and

Algorithm 1: The compose node of arity k , and what the root additionally requires. Line 2 is the binding of §9: without it the node would verify *some* proof rather than a proof of this machine.

Input: child proofs $\pi_0, \dots, \pi_{k-1}$ with public values $\text{pv}_0, \dots, \text{pv}_{k-1}$ and membership paths $\text{path}_0, \dots, \text{path}_{k-1}$; a claimed allowlist root root

Output: the node’s public values pv , or $\perp$

```

1 for  $i \leftarrow 0$  to  $k - 1$  do
2   if  $\neg \text{MerkleVerify}(\text{root}, \text{vk}_i, \text{path}_i)$  then return  $\perp$            // the child’s key is admissible
3   if  $\neg \text{Verify}(\text{vk}_i, \text{pv}_i, \pi_i)$  then return  $\perp$ 
4 if  $\exists i : \text{root}_i \neq \text{root}$  then return  $\perp$            // children agree with the root they were checked
   against
5 if  $\text{vk}_i, h_i, \text{code}_i$  not all equal over  $i$  then return  $\perp$ 
6 for  $i \leftarrow 0$  to  $k - 2$  do
7   if  $\text{exit}_i \neq \text{entry}_{i+1}$  then return  $\perp$            // ranges adjacent
8   if  $\text{cur}_{\text{out},i} \neq \text{cur}_{\text{in},i+1}$  then return  $\perp$            // cursors tile
9  $D \leftarrow \sum_i D_i$            // digests accumulate
10  $\text{pv} \leftarrow (\text{vk}, \text{entry}_0, \text{exit}_{k-1}, \text{cur}_{\text{in},0}, \text{cur}_{\text{out},k-1}, D, h, \text{root})$ 
11 if this node is the root then
12   if  $\text{cur}_{\text{in}}, \text{cur}_{\text{out}}$  non-empty  $\vee D \neq 0 \vee \text{code} \neq 0$  then return  $\perp$ 
13 return  $\text{pv}$ 

```

$\text{cur}_{\text{out},i} = \text{cur}_{\text{in},i+1}$, and the output satisfies $\text{entry} = \text{entry}_0$, $\text{exit} = \text{exit}_{k-1}$, $\text{cur}_{\text{in}} = \text{cur}_{\text{in},0}$, $\text{cur}_{\text{out}} = \text{cur}_{\text{out},k-1}$ and $D = \sum_i D_i$.

Leaves and the root. A leaf node proves the corresponding relation for a single child, with one difference that matters: its child is a core shard proof, which verifies against the guest’s vk rather than against a key of the allowlist. It is the leaf’s own key that the allowlist must contain, and its parent is what checks that. The root additionally requires cur_{in} and cur_{out} to be the empty cursors, D to be the identity and the exit code to be zero, which is what closes the memory argument and the cross-shard digest over the whole execution and admits only the halts the relation of Section 1.1 accepts. Algorithm 1 states the same checks as the procedure a compose node runs, with the root’s extra conditions on the last line.

The tree merges contiguous shard *ranges* as soon as they are complete rather than level by level, which shortens the tail after the last core shard from four serial composes to two; that tail is one of the two components of the gap to linear scaling in §11.5. Because a range is reduced the moment it is complete, the tree climbs on the workers that are still proving core shards, and on the deployed configuration the root is finished before the last shard’s proof is collected. A leaf is never itself the root: an execution of a single shard closes through a compose of arity one, which costs one more node and keeps the leaf programs to the open classes that the enumeration below covers. Arity is capped at three by the recursion units’ height budget; arity four was built and measured, and it was neutral (41 nodes replaced by 27, -0.7% of wall time, an unchanged tail), so it does not pay for the key regeneration it would force.

Shrink and wrap. The root proof is *shrunk* to a fixed shape and then *wrapped* on the outer ring: a jagged proof whose dense polynomial is committed with a FRI-style BaseFold instantiation over the same field but with a 254-bit hash, folding one variable per round under query grinding down to a small final polynomial. The artifact report estimates 534 KiB expected and 963 KiB worst case for the checked-in 16-bit-lookup-grinding schedule; these are configuration estimates rather than measurements of the zero-grinding baseline in Section 11. The wrap proof is what a Groth16 verifier on BN254, generated with **gnark** [Con24], consumes when an on-chain proof is required. For the block-proving service and throughout this paper, the compressed inner-ring proof itself is the published artefact.

Verifying-key binding. Every recursion program has a verifying key. The statement of Section 1.1 is only about the advertised machine if the verifier can tell that the leaf it is looking at verified a *core* proof

Algorithm 2: Enumerating the admissible leaf keys. The loop bounds come from the machine’s unit clusters and from the bucket rule of §7.5; no guest is executed. For the deployed machine it returns about $6.4 \cdot 10^3$ keys.

Input: unit clusters $\mathcal{C}$; bucket sets $\mathcal{B}_{\text{prep}}, \mathcal{B}_{\text{main}}$; padding rule $\text{pad}(\cdot)$
Output: the allowlist root root

```

1  $K \leftarrow \emptyset$ 
2 foreach cluster  $c \in \mathcal{C}$  do
3   foreach  $b_p \in \mathcal{B}_{\text{prep}}, b_m \in \mathcal{B}_{\text{main}}$  do
4      $n \leftarrow \text{pad}(b_m)$  // a function of the bucket, not of the real count
5     foreach  $\text{first} \in \{\text{true}, \text{false}\}$  do
6        $\sigma \leftarrow (c, \text{first}, b_p, b_m, n)$  // the shape a leaf can be asked to verify
7        $K \leftarrow K \cup \{\text{vk}(\text{CompileLeaf}(\sigma))\}$ 
8 return  $\text{MerkleRoot}(K)$ 

```

of that machine and that every compose node ran the compose program: the compose program therefore checks that its children’s keys belong to a Merkle allowlist, and the root of that allowlist is checked against the one compiled into the verifier. Which verifier matters: inside the tree the root is a witness the prover supplies, so the in-circuit checks establish only that every child’s key lies in a tree with *that* root. The root is therefore carried out of the tree as a public value of the compressed proof and, for the wrapped proof, as a public input of the wrap circuit, where the on-chain verifier supplies the published root rather than accepting one from its caller. It was not always so, and the gap is recorded in Table 12. The recursion machine is outside the extraction of Section 10, so Theorem 8.1 takes this binding as a hypothesis. It is part of the protocol: a proof produced by a prover configured to skip the in-circuit check has a different compose key and is rejected (Section 10 reports finding this configuration in the block-proving service). The allowlist is regenerated whenever the arithmetisation changes, and it is now enumerated for the leaf programs as well as the compose ones, which is what makes it a statement about the machine rather than about one guest.

Enumerating the leaf keys. A leaf program is determined by the geometry of the shard proof it verifies, and Section 7.5 quantises that geometry: the committed area of each round is a bucket rather than an exact cell count, and the padding-column count is a function of the bucket alone. What remains is the shard’s chip set, which is one of the machine’s fixed clusters, and the two bucket indices. Enumerating those triples and building one program per class gives, without running a single guest, every leaf key a prover can produce for a shard whose chip set is one of those clusters: about $6.4 \cdot 10^3$ keys for the deployed machine. The scope of that statement is the cluster list. A *cluster* is one of the chip-set combinations the machine’s shape configuration declares, curated from the combinations that arise in the workloads it is built for, not an exhaustive partition of the chip powerset. So the enumeration is exhaustive given the list, and the allowlist covers an arbitrary guest only to the extent that the list does; §1.3 says the same thing from the other side. What that buys is bounded on the right axis: a shard whose chip set falls outside the list produces a key the allowlist does not contain, and a verifier that requires membership refuses it. The failure is of completeness, not of soundness, and it is loud.

Why not sample the keys. Before this the leaf keys were *sampled*: a corpus of executions was proved and the keys they happened to produce were collected, which covers the workload that was sampled and leaves the next shard shape one uncollected key away from a refusal. A verifier configured to require membership on a sampled allowlist did exactly that in the block-proving service, which is how the omission was found. The enumeration is checked against the sampled corpus, class by class, and against the programs the classes generate. Algorithm 2 states the procedure; every quantity it ranges over is fixed by the machine and the committed geometry, so it terminates without consulting any execution.

10 The verification flow and its findings

The soundness argument of Sections 6–8 establishes that an accepting proof implies the existence of a trace satisfying every constraint. Two further questions decide whether that trace is an execution of the advertised machine: does the executor implement the ISA, and does each constrained unit admit exactly one output behaviour for a fixed input? We address both with a specification-derived conformance suite against an independent reference, an executable formal model, and per-unit uniqueness theorems. This section reports the verification artifacts of §E.4: the suites and their reference results, the formal model and its two checks, the extracted theorems with the tactic that discharges them and the list of those still open, and the defects register of Table 12. A deployer reads them to decide what has been established for a release and what remains conditional.

10.1 ISA conformance

Vectors from the specification. The MIPS32 architecture manual’s encoding tables were transcribed into a machine-readable table of 77 instructions (all integer, shift, branch, jump, load/store, bit-manipulation and conditional-move forms the guest supports), reproduced with the unit that proves each one in Appendix B. For each instruction a generator emits ten programs with pseudo-random operands, boundary values and the delay-slot placements the instruction admits, assembled with `11vm-mc`; the reference is the Unicorn/QEMU MIPS32 emulator [QV15], which shares no code with ZIREN’s own, and each vector compares the general registers, HI/L0 and the touched memory after execution. All 770 vectors agree, on both the interpreter and the JIT.

Shared instruction suites. The generated vectors cover the encoding space broadly but shallowly. A second collection contains fifteen hand-written suites, 144 cases in all, for instructions with difficult MIPS32 semantics: signed and unsigned division, multiply-accumulate pairs, bit-field and byte-order instructions, rotates, and count-leading-one and count-leading-zero instructions. One crate supplies case data to both the emulator and the prover tests, so every case is executed and proved against the same result. The generated vectors are shared in the same way. A regression in only the emulator or prover therefore appears as a disagreement rather than two tests drifting together.

The Cannon suite. The third suite is independent of us. Optimism’s Cannon ships a suite of hand-written MIPS test programs [Opt23], written by the fault-proof project whose ISA semantics a hybrid deployment must match (§E.3); they exercise the machine against another team’s reading of the architecture manual rather than against our own generator or our own regressions. A harness byte-swaps them to little-endian and adapts the halt convention; the 49 programs applicable to a little-endian guest all pass.

From vectors to proofs. 224 of the generated programs and 48 of the 49 Cannon programs were also proved end to end and verified. The single exception halts with a non-zero exit code, which the emulator accepts and the arithmetisation does not admit as a valid execution to prove. On everything else the arithmetisation accepts every behaviour the reference exhibits; the next subsection is the converse direction.

Executable formal model. A Lean 4 model of the ISA was written independently of ZIREN’s emulator: a decoder transcribed from the encoding tables, an interpreter with the delay-slot program-counter pair and little-endian byte memory, and an ALU. Two checks connect it to the implementation, both discharged by `native_decide`: the decoder agrees with the emulator’s decoder on all 2275 distinct instruction words appearing in the vectors, and the model reproduces the reference emulator’s results on all 770 vectors. ZIREN’s two emulator implementations are additionally run twice on every vector with a digest of the emitted event records; the digests agree between runs and between the interpreter and the JIT, and this is evidence of reproducibility rather than a proof of determinism.

10.2 Determinism of the units

What is proved. For a unit U with constraint system C_U over a row w of columns, and with *inputs* $I_U(w)$ (the values the row receives on its buses: the fetched instruction, the register and memory values read, the incoming state) and *outputs* $O_U(w)$ (the values it sends: the register and memory values written, the outgoing state), determinism is the statement

$$\forall w, w'. C_U(w) \wedge C_U(w') \wedge I_U(w) = I_U(w') \Rightarrow O_U(w) = O_U(w').$$

The statement is Definition 2.4, with byte-table lookups lowered to range constraints or to calls of an abstract deterministic table.

Theorem 6.5 is what makes a satisfying trace unique given the determinism of every unit. The theorems below are its premises, machine-checked for the units the automation closes and stated with an explicit gap for the rest.

Extraction. The theorems are not written by hand. The same Rust constraint description that the prover evaluates and the verifier checks is evaluated once with symbolic columns, the resulting polynomial constraints, range facts and bus interactions are lowered to a first-order statement, and a generator emits one Lean 4 file per unit: a record type for the row, the constraints as a conjunction, the input and output projections with their provenance, and the theorem above. Selector-specialised units (one theorem per opcode selector) and an *is_real* = 1 specialisation keep each statement about real rows of one opcode. The extraction covers every unit of the core machine catalogued in Appendix C: 106 determinism theorems.

Proof automation. A theorem is closed by a tactic that lifts the statement from $\mathbb{F}$ to the integers: every column is replaced by an integer with the bound its range fact gives (a byte, a 16-bit limb, a bit, or $[0, p)$ when unconstrained), every field equation becomes an exact integer equation once its slack is shown to vanish under those bounds, and the remaining goal is linear arithmetic over bounded integers, decided by Lean 4’s *omega*. Between the lifting and the decision the tactic runs the propagation loop of Picus inside the proof assistant: it proves corresponding columns of w and w' equal one pair at a time, substitutes, resolves selector bits to constants, and case-splits only on selector bits already known to agree. The automation runs with a fixed wall-clock budget per theorem.

Coverage. At the time of writing 17 of the 106 theorems are closed (Figure 8, by unit family). 22 were attempted and remain open: integer units whose constraints multiply by 2^{24} or 2^{32} and are only meaningful modulo p (multiply, the conditional moves, the memory-argument units), which the integer lifting does not model, or whose selector case structure exceeds the split budget. 41 are not attempted by the generator: 24 are accelerator units above the size threshold (the same theorem at 1 000–5 600 conjuncts), and 17 are units whose constraints contain comparison case-splits from the byte-table summaries, which the current tactic does not handle. 26 had not finished their attempt when the numbers were taken. The syscall unit, which was under-determined as first extracted (next paragraph), closes after the bus change; the count above is therefore for the constraint system with that change applied. Two units separate it from the system measured in Section 11: that one, and the hash unit, whose round schedule was corrected afterwards (Table 12). All theorem statements compile; open ones carry an explicit *sorry* and are listed in the artifact.

10.3 A negative control for the height claims

The suites above test that the machine accepts what it should. A verification flow also needs the opposite evidence: that the argument *rejects* a trace whose declared shape is a lie. This matters here because the shape is not incidental. The verifying key is a function of the chip set (§9), the committed geometry is quantised from the declared heights (§7.5), and the enumeration of admissible leaf keys rests on both. If a prover could declare a shape it did not execute, the enumeration would be a statement about nothing.

A harness therefore proves real shard proofs and then tampers with the claimed heights before verification, in eighteen cases across two guests, an arithmetic one and a hash-heavy one. The forgeries are over- and under-claims of a unit’s height, lies about which units are active, tampering with the column count and tampering with the row count, each injected at the stage that would carry it: the degree mask alone, the

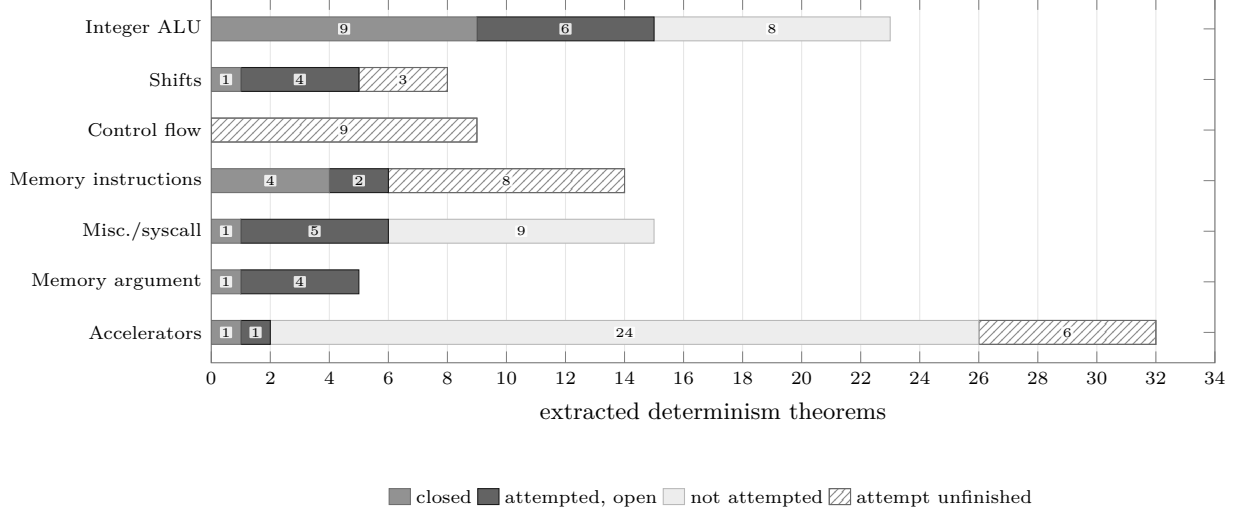


Figure 8: The current artifact closes 17 of 106 extracted unit and opcode-selector determinism statements. The remaining statements are open, skipped because of accelerator size or comparison case splits, or unfinished under the fixed automation budget.

Fiat–Shamir prologue alone, and both together. Every case is rejected, and the interest is in which check rejects it, because no single mechanism covers the space.

- Height and activity lies that reach the degree mask are caught by the **LogUp-GKR** last-layer reconstruction, as a numerator or denominator mismatch.
- Lies confined to the transcript are caught by the grinding witness, because the Fiat–Shamir prologue observes the declared heights, so a changed height invalidates the proof of work.
- A tampered column count is caught by the jagged verifier, and a tampered row count by the preprocessed round, which takes the row count from the verifying key rather than from the prover.

One case in the harness is a control on the control: a degree-only over-claim that an earlier configuration allowed to survive, because the reconstruction could be disabled. That configuration no longer exists, the anchor is unconditional, and the case now expects rejection. A negative control whose negative case cannot fail is not a control, which is why it is reported here rather than removed.

10.4 Findings

The flow surfaced nine defects, and their discovery methods distinguish three classes. Three determinism-theorem findings are under-constraints, where the constraints permit more than one value. Three end-to-end proving findings are integration mistakes between components that use different conventions. Three line-by-line comparison findings are omitted checks, a class that neither honest tests nor local determinism statements can detect.

That distinction is worth stating plainly, because it bounds what the first two techniques can establish. A determinism theorem asks whether the constraints present admit two behaviours; it is silent about a constraint that was never written, since its absence makes no theorem false. Testing is silent for the same reason: an omitted check breaks nothing that an honest execution does, so no test fails. The completeness flag left unpinned at the terminal stages is the sharpest example. Every honest proof sets it, so every test passes and every extracted theorem holds; only a reader asking what enforces it finds that nothing does. Differential reading against an independent implementation of the same protocol is the cheapest way we know to find that class, and it is the reason we report it as a third technique rather than as incidental review.

Appendix F tabulates all nine. One is described in detail here, because it is the one the extracted theorems found unaided and it shows what such a theorem does and does not say.

Syscall argument half-words. The syscall unit receives two reduced 32-bit arguments from the instruction that raised the syscall and witnesses each of them, written $a \in \mathbb{F}$ here, as two 16-bit half-words $a_{\text{lo}}, a_{\text{hi}}$ constrained by $a_{\text{lo}} + 2^{16}a_{\text{hi}} = a$ in $\mathbb{F}$ and two 16-bit range checks. Because $2^{32} > p$, this does not determine the half-words: for every $a \in \mathbb{F}$ both the decomposition of a and that of $a + p$ satisfy the constraints, since $2p < 2^{32}$. The extracted theorem for the unit is false and the automation reports exactly these four half-word columns as open. Whether the ambiguity is exploitable depends on the consumer: an accelerator that re-reduces the half-words modulo p sees the same field element either way, while one that uses them as an exact 32-bit pointer does not. The finding is a specification gap between the two sides of the syscall bus. The remedy, applied after the finding, is to carry the half-words rather than the reduced sum on the bus, together with the unit’s Linux selector; with that change the unit’s determinism theorem closes in under a minute. The change alters the constraints of two units and therefore the verifying keys, so it is staged with the next key-changing release rather than deployed alone.

11 Evaluation

This section reports two metrics in one deliberately narrow environment. *Emulator efficiency* is the rate at which the software emulator turns a guest program into the prover’s input record, together with the associated host memory and processor time (§11.2). *Prover efficiency* is the rate at which that record becomes a proof, together with its trace area and proof size (§11.3). The environment uses one workload, one GPU model, one host and interleaved repetitions on the same input so that differences between configurations are attributable.

Measurement status. Every timing and proof-size result in this section was collected before the Poseidon2 schedule was corrected from 8 full plus 13 partial rounds to 8 full plus 20 partial rounds. The correction affects commitments, Fiat–Shamir and recursion. The measurements therefore compare engineering changes within the pre-correction implementation; they do not report the absolute performance or proof artifact of the corrected release. Section 11.7 records this and the workload limitations.

Deployment. The system runs continuously in production rather than only in this evaluation: one deployment proves Ethereum mainnet blocks as they arrive on commodity GPUs, and others, the GOAT Network among them, prove different programs on the same prover. What follows measures the block workload, because it is the one with a public reference point; no claim here depends on the others.

11.1 Experimental setup

Workload. The workload is Ethereum mainnet block 25 907 955 executed by a MIPS32 build of the **reth** execution client with a witness trie represented as an arena. We use it because this class of system is publicly compared on block proving and because its instruction mix represents hash-intensive computation: the guest hashes trie nodes, recomputes a Merkle root over state it does not hold and verifies signatures. It is not a workload-diverse benchmark, and §11.7 bounds the conclusions accordingly. A word-wise **memcmp**, zero-copy Keccak wrapper and arena representation of the witness trie reduce the block from 379 to 288 million cycles. Every single-GPU prover measurement uses that cached record, so all runs execute the same instruction stream. The multi-GPU measurements of §11.5 use three further blocks of 420, 530 and 912 M cycles.

Hardware. One NVIDIA RTX 5090 (32 GB) in a host that holds eight of them, with one AMD EPYC 9355 processor and 925 GB of memory; the multi-GPU measurements use four of the eight (driver 570.153.02, CUDA 12.8, Ubuntu 22.04). The prover is pinned to one GPU.

Metrics. *Wall-clock time* is the end-to-end proving-process time from reading the cached input to writing the compressed proof. It includes host verification of every shard proof and the compressed proof, which accounts for 2–3 s and would be omitted by a production service. *Emulation rate* is guest cycles divided by the emulator’s wall-clock time. *Proving throughput* is guest cycles divided by the whole proving process’s wall-clock time. The latter includes the former, so the rates are not directly comparable; neither is a clock frequency. *Trace area* is the sum over units of rows $\times$ columns, in cells, from the emulator’s shard-close

Table 5: In the evaluated pre-correction baseline, **LogUp-GKR** consumes 42% of measured kernel time and the **Global** unit consumes 24.5% of the 19.6 G-cell trace area. Kernel shares come from a ten-shard **nsys** sample; area is rows $\times$ columns at the baseline shard count.

Kernel group	share	Unit	area share
LogUp-GKR layers (fold-and-sum, transitions, first layer)	42%	Global (48 M rows $\times$ 100)	24.5%
Zerocheck, compiled constraint kernels	16.5%	LoadWord (51 M $\times$ 49)	12.7%
Zerocheck, interpreted (wide precompile units)	7.1%	StoreWord (46 M $\times$ 53)	12.6%
Zerocheck fold	2.3%	MemoryUnaligned (30 M $\times$ 60)	9.1%
WHIR commit and open (Merkle absorb, DFT)	19%	Branch (22 M $\times$ 56)	6.3%
Jagged reduction	8%	AddSubImm (37 M $\times$ 33)	6.2%
Other	5%	ShiftLeftImm (20 M $\times$ 56)	5.8%

census and the unit widths. Table 9, in Appendix C, gives the widths of the release this paper describes, while Table 5 uses those of the evaluated revision, which precede the changes of §11.4; the appendix says where the two differ. *Proof size* is the byte length of the serialised compressed proof. A ten-shard **nsys** sample attributes GPU kernel time to kernel groups.

Software revision. The implementation described throughout the paper is v2.0.0, whose Poseidon2 permutation uses 8 full and 20 partial rounds. The measurements below were taken on earlier revisions of that branch, before the round correction, each row of Table 10, in Appendix D, adding its change to the row above it. The GPU backend is a separate CUDA implementation of the same protocol. No raw per-run timing bundle is retained (§11.7).

Protocol. Each pair of configurations is measured in interleaved A/B/B/A order on the same GPU and the same input, and medians are reported; the spread over three repetitions was within ± 1 s of the median for every row of Table 10 and within ± 0.8 s for Table 6. The GPU under measurement was otherwise idle; a second, unrelated proving process ran on another GPU of the same host, so the host processor was not idle.

11.2 Emulator efficiency

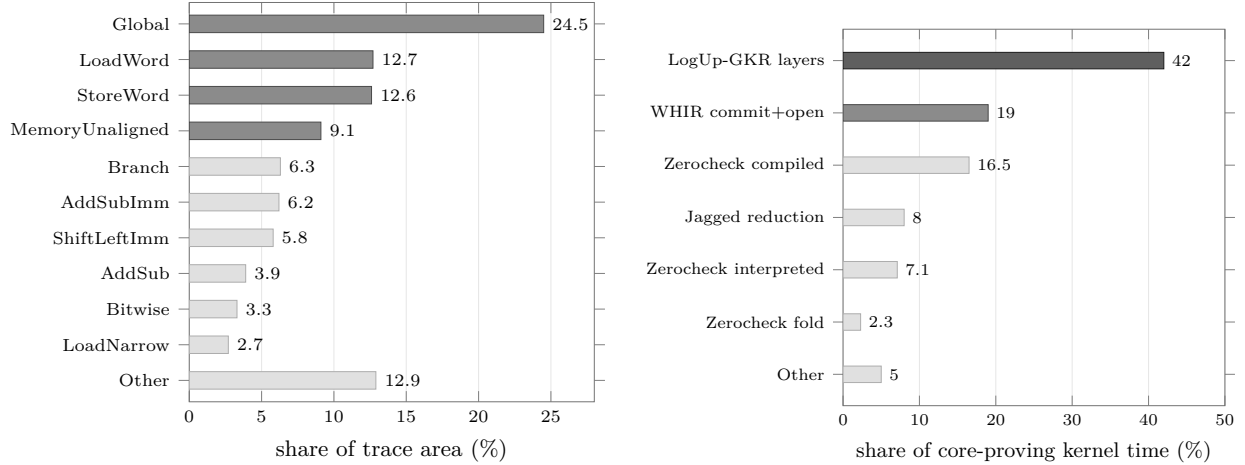
The emulator is the block’s front end: everything the prover consumes passes through it, and on a host driving several GPUs its cost, not the accelerator’s, is what binds (below). Its two implementations differ by an order of magnitude in rate. On the block of §11.1 the just-in-time compiler emulates the guest at roughly 130 MHz, while the interpreter runs at 5–20 MHz depending on whether it is recording events. That rate is the compiled emulator alone. A coordinator’s producer also writes each shard’s descriptor, and the last row of Table 7, in Appendix A, is that whole job at 6.1 s for a 420 M-cycle block, some 69 MHz. They produce the same records on the differential-test corpus (Section 3), so the measured rate difference does not change those tested witnesses. The compiled emulator is therefore some twenty times faster than proving and the interpreter roughly comparable to it, so on a single GPU emulation is not the binding cost.

It becomes binding when one host feeds several GPUs, because each shard is re-emulated by the worker that proves it: a worker’s replay and trace generation took about one second of host processor time per shard against half a second of GPU time. Appendix A reports the record-layout changes made under an explicit host budget, and the two rules they yield: an event should carry exactly what a column witnesses, and the process that coordinates several accelerators should never hold the whole execution.

Remark 11.1 (Optimisations not reflected in the tables). Two changes measured after the tables were taken are not reflected in them: wider clock fences, which reduce the shard count by 7%, and heavier query grinding with device-side early exit, which reduces the proof by 7%. Both were measured at a fixed revision and shard size, and neither moves wall-clock time.

11.3 Prover efficiency: where the cost is

Table 5 pairs the kernel-time profile with the area census; Figure 9 plots them side by side. Kernels account for approximately 340 ms of the 500 ms GPU time per shard, with launch gaps and host synchronisation



(a) Trace area by unit (19.6 G cells); **Global** and the three largest memory-instruction units hold 59%. (b) **LogUp-GKR** is the largest kernel group at 42%, ahead of **WHIR** commit and open at 19% (nsys, ten-shard sample).

Figure 9: The **Global** and three largest memory-instruction units hold 59% of trace area, while **LogUp-GKR** is the largest measured kernel group at 42%. Both panels describe the evaluated pre-correction baseline before the changes of §11.4.

making up the difference. The lookup argument is the largest kernel consumer, and its cost scales with $(row, interaction)$ pairs. This block has 7.8 G pairs and 19.6 G trace cells, suggesting that one pair costs approximately two cells. Memory-instruction units carry 29 interactions per row, including 21 byte-table lookups, and account for 48% of all pairs. In contrast, the **Global** unit holds one quarter of trace area but only 2.5% of pairs because every touched word costs two rows. The resulting cost model uses area for commitments and zerocheck, and pairs for lookup, with one pair weighted as two cells.

What the soundness target costs. The per-component target of \$8.4 is paid for in queries and in proof-of-work, and only the first is visible in the proof. Measured on the workload block over four cards of the eight-card host, with the other four carrying an unrelated proving service throughout, each schedule reached from one binary through its environment, and the arms interleaved: raising the query, **LogUp-GKR** and batching grinds from 16/16/8 to 22/22/14 bits left the block at 33.4s against 33.9s for the former schedule, and buying the same bits with queries instead ([133, 95, 91] at a 16-bit grind) cost 8.6%. Neither figure holds for a prover whose proof-of-work search is not proportional to 2^{bits} : the same schedule took 166s while the grinds ran on the host inside the per-shard parallelism, 123s of it the **LogUp-GKR** grind alone. A grinding bit is cheap for the honest prover only in the sense that the search is; it is not free, and it buys a bit of soundness at the cost of a factor of two in that search.

11.4 What reducing area buys

The cost model of §11.3 says where to cut: area for the commitment and the constraint argument, and $(row, interaction)$ pairs for the bus argument. Four changes follow from it, three that remove columns or interactions from the units the census names and one that raises the shard size, and together they take the same block from 74.1 to 67.1s on one GPU, a reduction of 9.4%. Two of them are worth naming because they are mechanisms rather than tuning: replacing a boolean sign decomposition and its witness columns in the cross-shard digest unit with three byte lookups, and dropping byte checks on memory-instruction accesses that another unit already enforces. Neither changes what any constraint reads, so the verifying keys and the proofs are unchanged. Appendix D gives the four measurements one at a time, and the two negative results that came with them: a unit narrowed below measurement noise, and a shard size that did not fit in device memory.

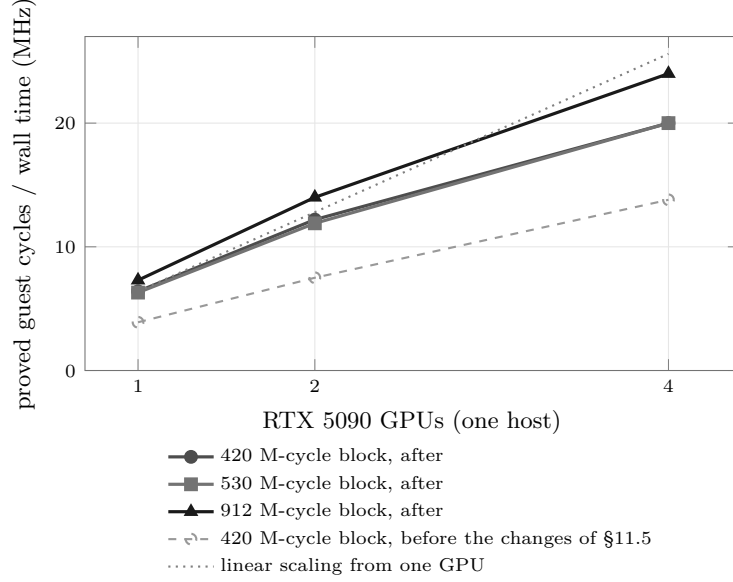


Figure 10: The pre-correction implementation reaches $3.1\text{--}3.3\times$ one-GPU throughput on four GPUs across the three block sizes. The dotted line is ideal linear scaling; the baseline series is the evaluated zero-lookup-grinding configuration. None of the plotted throughput values measures the corrected release: every curve is the pre-correction Poseidon2 schedule, whose keys and proofs the correction invalidates (§11.7), so the curves characterise scaling behaviour and not delivered performance.

11.5 Prover efficiency: memory traffic, and scaling across devices

The profile identified the **LogUp-GKR** circuit as the largest kernel consumer (42%). A byte-level profile of its layers showed that the first layer paired adjacent fractions across the most-significant index bit, so that short units were carried through every layer of the tallest unit and the traffic summed over layers was $5.2\times$ that of the first layer. Pairing on the least-significant bit instead lets each unit fold out after $\log_2$ of its own height. The change also frees enough memory for shards of twice the baseline size, which did not fit before it (§11.4). The two are therefore enabled together, and the figure below reports the pair. We do not attribute the result to the layer order alone: separating them needs a 2-by-2 ablation over old/new pairing and baseline/doubled shard size, and the old-pairing/doubled-shard cell is the one that did not fit in memory, so that cell would have to be reported as a failure rather than a time. Until that experiment exists the result stands as one combined configuration change. With both, the same block proves in 48.7s on one GPU, against 67.8s for the revision without either at the largest shard size it could run (-28% for the pair; the 67.1s of Table 10 is a different revision), with the proof unchanged at 617618 bytes.

Table 6 reports the scaling of that configuration from one to four GPUs on a 420 M-cycle block against the evaluated baseline of Table 10. The throughput columns give proved guest cycles per second of wall-clock time. Figure 10 plots throughput for three block sizes: 420, 530 and 912 M cycles. The larger blocks take 84.3/44.7/26.5s and 124.3/65.0/38.0s on one, two and four GPUs. Throughput varies by less than 20% across the three blocks. On the 420 M-cycle block it scales by $1.9\times$ from one GPU to two and by $3.1\times$ to four; the two larger blocks reach $3.2\times$ and $3.3\times$ at four. The change helps least at four GPUs because it shortens core proving, whose share falls as the recursion tail and start-up ramp grow. The remaining gap to linear scaling comes from the serial recursion tail and from the interval before the emulator has produced enough shards to occupy every GPU.

Where the GPU work sits. Three kinds of engineering carry the figures above, and they are worth separating because they act on different costs. *Fused kernels* act on memory traffic: the jagged fold and the round-polynomial evaluation it feeds are one kernel rather than two, so the folded values stay in registers and shared memory instead of making a round trip through device memory between them, and the same fusion is

Table 6: The lookup-layer and shard-size changes improve pre-correction throughput by $1.64\times$ on one GPU and $1.45\times$ on four GPUs for a 420 M-cycle block. Values are warm wall-clock times for three consecutive proofs; the instrumented proof size was 617 618 bytes.

GPUs	wall-clock (s)			throughput (MHz)		
	1	2	4	1	2	4
Before (evaluated baseline shards)	108.0	56.0	30.5	3.9	7.5	13.8
After (LSB pairing, doubled shards)	65.9	34.3	21.0	6.4	12.2	20.0
Speed-up	$1.64\times$	$1.63\times$	$1.45\times$			

applied to the layer transition and to the round over the mixed round-0 slab. A *cross-stage pipeline* acts on device idleness: the leaves and compose nodes of the reduce tree are proved in single-device child processes while core shards are still being proved (`core_multi_gpu::reduce_overlap`), with one tail worker for shrink and wrap, so the recursion tree is not a phase that begins when core proving ends: on the measured block the root is delivered before the last shard proof is. *Occupancy tuning* acts on the kernels themselves: the register budget is capped for the architecture of the card measured here, which trades spills against resident warps.

Underneath all three, field arithmetic rests on the inline-PTX Montgomery primitives of `sppark` [Sup25], which we use rather than wrote; the `KoalaBear` base and extension types are built on that layer, and no kernel of ours emits assembly of its own. We separate this deliberately: the performance reported here is a property of the whole stack, and the parts of the stack that are other people’s work are named as such.

11.6 Answering the question of Section 1

The opening question has three parts. *Is the prover’s cost attributable to identified choices?* The cost model of §11.3 names where the cost is, and the changes it suggested take the same block from 74.1 to 67.1 s, 9.4%, with the per-change measurements in Appendix D. A further 28% comes from the lookup-layer and shard-size pair of §11.5, which the section declines to separate for want of one cell of the ablation. Across those revisions the same block went from 74.1 to 48.7 s and four GPUs reached 20 MHz, but the corrected hash schedule remains unmeasured. Cross-system comparison is a different question, and this paper does not answer it. This paper reports what this prover costs and why, not how that compares: the figures other systems publish are self-reported, measured on different guests and mostly on clusters, and nothing here is a same-hardware baseline against any of them. *What assumptions support soundness?* The proximity-test schedule uses the unique-decoding regime and a union bound, but the measured zero-lookup-grinding baseline does not inherit the checked-in report’s 100-bit result. The random-oracle model, conditional extractor composition and unquantified structured-relation assumption for the cross-shard digest also remain. *How much of the constraint system is verified?* The emulator is tested against an independent reference and formal model, 17 of 106 extracted determinism statements are closed, trace uniqueness has explicit premises, and the recursion machine is not extracted.

11.7 Threats to validity

Internal validity. All same-hardware comparisons are interleaved and repeated, and the measured spread is small relative to the reported effects; the one effect within the noise is reported as neutral. The host processor was shared with an unrelated process during the single-GPU measurements, so host-side interference was not controlled. *Construct validity.* Every measurement used the pre-correction hash schedule whose partial-round count came from a higher S-box degree (§10.4). Correcting it adds seven partial rounds to commitments, transcripts and recursion, invalidates the corresponding keys and proofs, and changes absolute performance by an unmeasured amount. The internal ablations remain evidence about the mechanisms only to the extent that the correction does not change their relative costs. Wall-clock time includes host verification, which a production deployment omits. Trace area is a cost model; the kernel profile is the direct measurement. *External validity.* The primary workload is one Ethereum block. Other guests have different instruction mixes and per-unit area shares. No cross-system comparison is attempted, so nothing here establishes a ranking. No raw per-run timing bundle or exact 617 618-byte proof is retained in the current artifact.

12 Conclusion

We have presented the second generation of ZIREN as an end-to-end verifiable-computation system for MIPS32 execution. Its executor feeds opcode-specific arithmetic tables; its instruction frame and shared bus arguments connect those tables to program, state and memory; its Jagged-WHIR layer reduces columns of different heights to one batched opening; and its recursion and verifier turn shard proofs into one deployable artifact. Trace area, interaction count, proof size and accelerator throughput therefore describe one implemented CPU-GPU proof system, not a synthesisable processor.

We have also separated the evidence for this design into explicit layers. Conformance suites and an executable Lean 4 model test the emulator, while mechanically extracted statements expose determinism obligations in the constraint system. The current artifact closes 17 of 106 such statements and leaves recursion outside the extraction. At the proof-system layer, the unique-decoding accounting applies to the checked-in schedule, while end-to-end knowledge extraction remains conditional on the per-node and extractor-composition premises of Theorem 8.1. The cross-shard digest remains a separate structured-relation assumption.

The performance study has identified trace area, lookup interactions, layer order and shard size as distinct cost levers. The largest item it leaves on the table is the cross-shard digest: every word touched in a shard costs two rows of that unit, one quarter of the block’s area, and either a digest of the per-shard delta or a larger shard would reduce it, the latter bounded by the memory the lookup circuit’s first layer needs. Its measurements characterise the pre-correction implementation and one block-proving workload; they do not establish corrected-release performance or a same-hardware ranking against other systems. The next release should unify one runtime configuration, soundness report, key set, serialised proof and benchmark bundle after the Poseidon2 correction. Completing the composition proof, quantifying the digest assumption and closing the remaining trace-uniqueness premises are the corresponding proof priorities.

Acknowledgements

This system is built on work done by others, and two debts are large enough to name here rather than only in citations. The Plonky3 [Pol24] toolkit supplies the field, the Poseidon2 permutation [GKS23], the constraint interfaces and the transforms this implementation is written against. SP1 [Suc24, Suc25a] is the system this design learned the most from: the decomposition into one unit per opcode with a shared lookup argument, the elliptic-curve digest that carries interactions across shards, the shadow read that makes register accesses cheaper than memory accesses, and the shape of the recursion layer all follow it, and its just-in-time executor is the model for ours. Where this paper departs from either, the section that does so says why.

A The just-in-time executor

Section 3.4 states that the execution relation is realised twice and says what connects the two realisations. This appendix gives the mechanism of the second one, a just-in-time compiler for Linux x86-64 hosts, which is what a production prover runs. Nothing in the protocol depends on it: a platform without the compiler proves the same statements more slowly.

A.1 Two implementations, three jobs

The interpreter is a single loop over `execute_cycle`: fetch, dispatch on the opcode, apply the state transition, append the events the trace generator will consume. It implements every instruction, every syscall and every error path, and it is the only implementation that emits the full event record.

The compiler is used for the two jobs that dominate wall-clock time and need no events. The first is *whole-program execution*, when a caller wants only the output stream and the exit status. The second, which the multi-GPU prover depends on, is *minimal-trace production*: the coordinating process runs the guest to the end and emits, per shard, a chunk descriptor holding the pre-access value of every memory word the shard reads, the register records at its boundary, and the boundary itself. A worker then re-executes the shard from that descriptor under the interpreter and emits the events locally. The consequence is that no

event record crosses a process boundary and the coordinator holds one shard's state at a time, and also that the event-emitting path stays the interpreter's: the compiler never has to reproduce an event stream, only the state and the accounting a replay needs.

Availability is a build-time decision. The parent crate's build script emits `cfg(zkm_use_native_executor)` on Linux x86-64 without the profiling feature; elsewhere the flag is absent and every entry point stays on the interpreter, so a platform without the compiler is slower and not otherwise different.

A.2 Transpilation and the program cache

Compilation is one-shot and per program, not per basic block discovered at run time. The driver walks the program's instruction stream once, and for each instruction records the native offset that the guest program counter maps to, dispatches on the opcode to a lowering, and emits the epilogue that advances the clock and the program counter. The result is a native code buffer, mapped executable from an anonymous file, together with a jump table holding one native address per guest program counter. Control transfers inside the compiled program are table dispatches, so the compiled unit is the whole program and a branch never leaves it.

Two properties of the lowering are worth stating because they are what make it tractable. Delay slots are resolved statically: a branch or jump whose delay slot holds another branch or jump, or which ends the program, is rejected at compile time and the host falls back to the interpreter, so the only instruction that has to read a delayed target is the one immediately after a transfer. And an opcode the lowering does not model is a compile-time refusal rather than a run-time surprise: the driver returns the offending opcode and the caller falls back.

Compiled programs are cached process-wide. The key must be a hash of the entire instruction stream: the base address, the instruction count and, for every instruction, the opcode and all three operands with their immediate flags. A cheaper fingerprint is unsound in a way that does not announce itself: two guests that agree on a prefix and a suffix but differ in the middle collide, and the second one silently executes the first one's code. ZIREN hashes the whole stream for this reason, and a test asserts that changing any single instruction changes the key.

A.3 Machine state in host registers

The guest's register file is held in host registers for the duration of a run. There are 33 live guest registers (31 general-purpose plus HI and L0) and 16 vector registers offering 32 half-slots, so exactly one must live elsewhere: general registers 1 to 30 pack two per vector register into the first fifteen, HI and L0 take the halves of the sixteenth, and the return-address register takes a general-purpose host register of its own, because it is read and written by every call sequence and a half-slot access costs an insert or extract. Register 0 is not stored at all. The two registers that only `mmap`-style syscalls touch stay in the context structure in memory.

Three quantities are pinned to host registers alongside the guest state: the packed clock, the oracle tail pointer, and the remaining trace-area budget. All three are written back to the context around any call into the host and on exit, so the host never reads a stale copy.

Guest memory is a flat array with one sixteen-byte entry per guest word, a value, the timestamp and shard of its last access, and padding that keeps the index a shift of the address. The array is a shared mapping of an anonymous file with reservation disabled, so it is sparse and only touched pages are ever committed, and an unconstrained block is entered by mapping a private copy-on-write view of the same file and left by unmapping it, so the rollback that the paged executor performs with a per-address diff becomes one unmap. Two alignments in that entry are load-bearing. Its first twelve bytes are exactly the image of an oracle entry, so recording a pre-access value is one twelve-byte copy; and its timestamp and shard fields are the little-endian image of the packed clock register, so stamping an access is one eight-byte store.

A.4 What the compiled code accounts for

A compiled instruction is not just its state transition. To let a worker replay a shard, and to close shards where the interpreter would, the emitted code also performs the bookkeeping of Section 3:

- **Access stamps.** Every register the interpreter stamps is stamped here, with the same shard and the same sub-cycle position, at the highest position at which the instruction touches it.
- **The clock.** An instruction executes at clk and its accesses occupy $\text{clk} + 1, \dots, \text{clk} + 4$ by position, so the clock advances by five per instruction. The pinned register holds the shard and the clock as one word, which is what makes the memory stamp a single store.
- **The oracle.** A read or write of an address above the register window copies the entry as it stood before the access to the oracle tail, which is the descriptor the replaying worker will answer memory reads from.
- **Shard budgets.** The trace-area budget in its pinned register and the per-unit height budgets in the context are charged the exact amounts the interpreter’s accumulator charges, supplied per instruction by the compiler. Exhaustion of any one of them is the condition under which the interpreter would fence, and the compiled function returns a shard-fence status so the host seals the chunk and re-enters.

Because the budgets are charged in native code and checked on every instruction, the shard boundaries a compiled run produces are the boundaries an interpreted run would produce, which is what allows the two to be compared chunk by chunk.

A.5 What the interpreter keeps

The compiler deliberately implements no error path and no syscall. Every **SYSCALL**, every division by zero, every trap instruction, every misaligned or out-of-range address, and every operand from the lowering does not model is a *trap site*: the emitted code stores the guest program counter and jumps to a shared stub whose host handler synchronises the native state into the interpreter, runs that one instruction there, synchronises back and resumes the compiled code at the resulting program counter. The interpreter therefore owns the precompiles, the unconstrained blocks and every failure, and the compiler owns the arithmetic, memory and control flow that make up almost every instruction of a real workload.

This split is what keeps the compiler small enough to argue about. It is also why a syscall-dense guest sees less speedup than a compute-dense one: each trap is a state synchronisation in both directions.

A.6 Evidence that the two agree

Three checks connect the implementations, and none of them is a proof of equivalence for every program.

The producer’s output is compared against the interpreter’s directly: tests assert that the chunk descriptors a compiled run emits are byte-identical to those of an interpreted run of the same program, which covers the state, the oracle, the stamps and the shard boundaries in one comparison. The 770 specification vectors of Section 10 are executed on both implementations and against an independent reference emulator. And continuous integration compares digests of the emitted records across runs and across implementations on the tested corpus, which is evidence of determinism and of agreement there rather than everywhere.

Section 11.2 measures what the compiler buys: on the reference block it emulates the guest at roughly 130 MHz against the interpreter’s 5–20 MHz, depending on whether the interpreter is recording events.

A.7 The record pipeline under a host budget

Section 11.2 states that on a host feeding several accelerators the executor, not the accelerator, is what binds, because each shard is re-emulated by the worker that proves it. The event stream written by one shard replay was approximately 1 GB. Table 7 lists the changes made under an explicit host budget. Each preserves the represented execution semantics: the trace columns every constraint reads are unchanged, so the verifying keys and the proofs are unchanged. The record layout itself does change; the table below removes fields and narrows record widths, so each change is witness-equivalent rather than byte-identical, and it is the event record, not the committed trace, whose bytes move.

Two rules follow. An event should carry exactly what a column witnesses: three of the seven rows remove fields that only a debug assertion or an unused code path had kept alive; on the shared register records alone this is 44 bytes for every instruction event, one write record and two read records. And the process that

Table 7: Emulator and record pipeline under a host memory and processor budget. Sizes are per event or per shard of the block of §11.1; times are host-side, single process, three repetitions.

Change	before	after	effect
Per-cycle event: keep clock, pc pair and exit code only (no CPU table reads the rest)	184 B	20 B	event stream per shard 1,000 MB → 462 MB; replay 700 ms → 435 ms
Memory-instruction event: drop the three fields no column reads	168 B	136 B	every load/store event
Register-access record: carry only the witnessed values (no shard fields; one arm for read/write)	48 / 24 B	20 / 16 B	every instruction event
Memory oracle shipped to a worker: pre-access value, stamp and shard as one byte run	24 B, 25 MB	12 B, 12 MB	per access, per shard; serialise 6–7 → 1 ms
Coordinator producer without checkpoints: no state clone or first-touch map per shard	22.9 s	20.4 s	per block; state $O(\text{shard})$
Native (JIT) producer in the coordinator	12.3 s	6.1 s	per 420 M-cycle block
Flat guest memory: one 16-byte entry per word in a sparse mapping; untouched pages cost nothing; unconstrained blocks are copy-on-write views	paged table	flat, sparse	no per-block memory diff

coordinates several GPUs should never hold the whole execution: the coordinator’s resident state is one shard’s oracle and the guest image, so the block size is not bounded by host memory; only the shard size is bounded, by the accelerator’s memory (§11.4). Both rules apply to any zkVM whose prover is fed by a re-executing host.

B The guest instruction set

This appendix lists the MIPS32 instructions a guest may execute, which is the set the relation of §1.1 quantifies over. It is also the table from which the conformance corpus of §10 is generated: each row produces ten programs with pseudo-random operands, boundary values and the delay-slot placements the instruction admits, giving the 770 vectors compared against an independent emulator. The last column names the unit that arithmetises the instruction, so the table fixes which rows of Appendix C a program can occupy, and with it the chip set that Section 9 enumerates over.

Three conventions are worth noting. HI and LO occupy the register file as registers 32 and 33, so the four moves between them and the general registers are additions of an immediate zero rather than a unit of their own. SYNC, SYNCI and PREF are accepted and retire as no-ops, for which the same unit suffices. SYSCALL is absent from the table because it is the boundary of the machine rather than a computation within it; it is arithmetised by `SyscallInstrs` and its interface is Section 4.

Table 8: The 77 guest instructions, their primary encoding fields and the unit that proves each. *Op* is bits 31–26 and *func* bits 5–0; a dash marks a field the format does not use, and the immediate forms are distinguished from the register forms by *Op* alone. Semantics are as transcribed from the architecture manual into the specification table that generates the conformance corpus.

Mnemonic	Op	func	Family	Semantics	Unit
add	000000	100000	Integer arithmetic	$rd = rs + rt$	AddSub
addu	000000	100001		$rd = rs + rt$	AddSub
addi	001000	--		$rt = rs + \text{sext}(\text{imm})$	AddSubImm
addiu	001001	--		$rt = rs + \text{sext}(\text{imm})$	AddSubImm
sub	000000	100010		$rd = rs - rt$	AddSub
subu	000000	100011		$rd = rs - rt$	AddSub
mul	011100	000010		$rd = rs * rt$	Mul
mult	000000	011000		$(hi, lo) = rs * rt$	Mul
multu	000000	011001		$(hi, lo) = rs * rt$	Mul
div	000000	011010		$(hi, lo) = (rs\%rt, rs/rt),$ signed	DivRem
divu	000000	011011		$(hi, lo) = (rs\%rt, rs/rt),$ unsigned	DivRem

continued on the next page

Table 8, continued from the previous page

Mnemonic	Op	func	Family	Semantics	Unit
madd	011100	000000	Comparison and logic	$(hi, lo) = (hi, lo) + rs * rt$ (signed)	MiscInstrs
maddu	011100	000001		$(hi, lo) = rs * rt + (hi, lo)$	MiscInstrs
msub	011100	000100		$(hi, lo) = (hi, lo) - rs * rt$ (signed)	MiscInstrs
msubu	011100	000101		$(hi, lo) = (hi, lo) - rs * rt$	MiscInstrs
clo	011100	100001		$rd = \text{count_leading_ones}(rs)$	CloClz
clz	011100	100000		$rd = \text{count_leading_zeros}(rs)$	CloClz
slt	000000	101010		$rd = rs < rt$	Lt
sltu	000000	101011		$rd = rs < rt$	Lt
slti	001010	--		$rt = rs < \text{sext}(imm)$	LtImm
sltiu	001011	--		$rt = rs < \text{sext}(imm)$	LtImm
and	000000	100100		$rd = rs \& rt$	Bitwise
andi	001100	--		$rt = rs \& \text{zext}(imm)$	BitwiseImm
or	000000	100101		$rd = rs rt$	Bitwise
ori	001101	--		$rd = rs \text{zext}(imm)$	BitwiseImm
xor	000000	100110		$rd = rs \wedge rt$	Bitwise
xori	001110	--		$rd = rs \wedge \text{zext}(imm)$	BitwiseImm
nor	000000	100111		$rd = !rs rt$	Bitwise
lui	001111	--		$rt = imm \ll 16$	AddSubImm
sll	000000	000000	Shifts and bit fields	$rd = rt \ll sa$	ShiftLeftImm
sllv	000000	000100		$rd = rt \ll rs[4:0]$	ShiftLeft
srl	000000	000010		$rd = rt \gg sa$	ShiftRightImm
srlv	000000	000110		$rd = rt \gg rs[4:0]$	ShiftRight
sra	000000	000011		$rd = rt \gg sa$	ShiftRightImm
srav	000000	000111		$rd = rt \gg rs[4:0]$	ShiftRight
rotr	000000	000010		$rd = \text{rotate_right}(rt, sa)$	ShiftRightImm
rotrv	000000	000110		$rd = \text{rotate_right}(rt, rs[4:0])$	ShiftRight
ext	011111	000000		$rt = rs[\text{msbd} + \text{lsb}.. \text{lsb}]$	MiscInstrs
ins	011111	000100		$rt = rt[32:\text{msb}+1] $ $rs[\text{msb}+1-\text{lsb} : 0] $ $rt[\text{lsb}-1:0]$	MiscInstrs
wsbh	011111	100000	Control transfer	$rd = \text{swaphalf}(rt)$	MovCond
seb	011111	100000		$rd = \text{signExtend}(rt[7..0])$	MiscInstrs
seh	011111	100000		$rd = \text{signExtend}(rt[15..0])$	MiscInstrs
beq	000100	--		$PC = PC + \text{sext}(\text{offset} \ll 2), \text{ if } rs == rt$	Branch
bne	000101	--		$PC = PC + \text{sext}(\text{offset} \ll 2), \text{ if } rs \neq rt$	Branch
bgez	000001	--		$PC = PC + \text{sext}(\text{offset} \ll 2), \text{ if } rs \geq 0$	Branch
bgtz	000111	--		$PC = PC + \text{sext}(\text{offset} \ll 2), \text{ if } rs > 0$	Branch
blez	000110	--		$PC = PC + \text{sext}(\text{offset} \ll 2), \text{ if } rs \leq 0$	Branch
bltz	000001	--		$PC = PC + \text{sext}(\text{offset} \ll 2), \text{ if } rs < 0$	Branch
j	000010	--		$PC = PC[\text{GPRLEN}-1..28] $ $\text{instr_index} 00$	Jump
jal	000011	--		$r31 = PC + 8, PC =$ $PC[\text{GPRLEN}-1..28] $ $\text{instr_index} 00$	Jump
jr	000000	001000		$PC = rs$	Jump
jalr	000000	001001		$rd = PC + 8, PC = rs$	Jump
bal	000001	--		$RA = PC + 8, PC = PC +$ $\text{sign_extend}(\text{offset} 00)$	Jump
teq	000000	110100		$\text{trap, if } rs == rt$	MiscInstrs
lb	100000	--	Aligned memory	$rt = \text{sext}(\text{mem_byte}(\text{base} + \text{offset}))$	LoadNarrow
lbu	100100	--		$rt = \text{zext}(\text{mem_byte}(\text{base} + \text{offset}))$	LoadNarrow

continued on the next page

Table 8, continued from the previous page

Mnemonic	Op	func	Family	Semantics	Unit
lh	100001	--		rt = sext(mem_halfword(base + offset))	LoadNarrow
lhu	100101	--		rt = zext(mem_halfword(base + offset))	LoadNarrow
lw	100011	--		rt = mem_word(base + offset)	LoadWord
ll	110000	--		rt = mem_word(base + offset)	LoadWord
sb	101000	--		mem_byte(base + offset) = rt	StoreNarrow
sh	101001	--		mem_halfword(base + offset) = rt	StoreNarrow
sw	101011	--		mem_word(base + offset) = rt	StoreWord
sc	111000	--		mem_word(base + offset) = rt, rt = 1, if atomic update, else rt = 0	StoreWord
lwl	100010	--	Unaligned memory	rt = rt merge most significant part of mem(base+offset)	MemoryUnaligned
lwr	100110	--		rt = rt merge least significant part of mem(base+offset)	MemoryUnaligned
swl	101010	--		store most significant part of rt	MemoryUnaligned
swr	101110	--		store least significant part of rt	MemoryUnaligned
mfhi	000000	010000	Moves and no-ops	rd = hi	AddSubImm
mflo	000000	010010		rd = lo	AddSubImm
mthi	000000	010001		hi = rs	AddSubImm
mtlo	000000	010011		lo = rs	AddSubImm
movn	000000	001011		rd = rs, if rt != 0	MovCond
movz	000000	001010		rd = rs, if rt == 0	MovCond
sync	000000	001111		sync (nop)	AddSubImm
synci	000001	--		sync (nop)	AddSubImm
pref	110011	--		prefetch(nop)	AddSubImm

C The unit set of the core machine

Section 6 describes the arithmetisation as a set of units, a frame and two shared arguments, without depending on which units exist. This appendix is the catalogue of the units themselves. The widths matter only to the area accounting: the protocol of Section 5 sees the whole shard as one dense vector, so a unit may be added, widened or removed without changing anything above the jagged reduction.

Every width in Table 9 was read from the machine at the commit this paper describes (§11.1) rather than from a census, and the table lists all 62 units rather than the ones that dominate the area. That pins it to one revision, which the evaluation’s tables are not: the measurements of §11.4 were taken on earlier revisions, so a few units differ between them and this table. The **Global** unit is the clearest case. It carries 65 columns here, against the 100 of the census in §11.3 and the 58 that the change of §11.4 reduced it to; work landed between that measurement and this commit. Where the two disagree, the evaluation’s numbers belong to the revision each table names and this one belongs to the commit.

D Per-change measurements

Section 11.4 reports one configuration end to end. This appendix records the four changes one at a time, on the revision and workload of §11.1, together with the two that did not pay. Numbers of this kind age with the revision they were taken on, and none of them is a property of the protocol.

The cost model of §11.3 suggested the changes in Table 10. Replacing the 30-bit boolean sign decomposition, eight offset bits and seven chord-witness columns of the **Global** unit with three byte lookups removed 42 columns from 48M rows and 3.9% of wall-clock time. The instrumented compressed-proof measurement remained 617618 bytes, while the core proofs, some 10^8 bytes per block, shrank by 3.4%. Removing redundant

Table 9: The 62 units of the core machine with their main-trace widths (columns over $\mathbb{F}$), read from the machine of the release named in §11.1; a second figure is a preprocessed width. The immediate (**Imm**) variants take the second operand from the instruction word. **MovCond** proves **movn**, **movz** and **wsbh**, and **SysLinux** the Linux-ABI syscalls of Section 4; Appendix B gives the instruction-to-unit map.

Family	Units (width)
Integer ALU	AddSub (36), AddSubImm (33), Bitwise (37), BitwiseImm (33), Mul (74), DivRem (163), Lt (50), LtImm (47), CloClz (93)
Shifts	ShiftLeft (62), ShiftLeftImm (45), ShiftRight (89), ShiftRightImm (83)
Control flow	Branch (56), Jump (57)
Memory instructions	LoadWord (48), LoadNarrow (59), StoreWord (52), StoreNarrow (55), MemoryUnaligned (59)
Misc. / syscall	MiscInstrs (432), MovCond (49), SyscallInstrs (68), SyscallCore (11), SyscallPrecompile (11), SysLinux (100)
Memory argument	MemoryLocal (80), MemoryBump (12), MemoryGlobalInit (152), MemoryGlobalFinalize (152), Global (65)
Tables	Program (1, +14), Byte (10, +12), Range (1, +2)
Hash precompiles	ShaExtend (166) and control (4), ShaCompress (249) and control (69), KeccakSponge (2637) and control (1095), Poseidon2Permute (571)
Curve precompiles	add (1685) and double (1666) for Secp256k1 , Secp256r1 and Bn254 , decompress (1040) for the first two; Bls12381 add (2533), double (2506), decompress (1613); Ed25519 add (1433), decompress (1166)
Field precompiles	Bls12381Fp (512), Bls12381Fp2AddSub (1014), Bls12381Fp2Mul (1773), Bn254Fp (344), Bn254Fp2AddSub (678), Bn254Fp2Mul (1181), Uint256MulMod (417), U256XU2048Mul (2627)

Table 10: The four cumulative changes reduce the pre-correction single-GPU time from 74.1 to 67.1s on block 25 907 955. Values are medians of interleaved, host-verified runs; each row adds one change to the previous row.

Configuration	shards	wall-clock (s)	vs. previous
Evaluated baseline, no lookup grinding	130	74.1	
+ Global 100 → 58 columns (§6.4)	130	71.2	−3.9%
+ memory-instruction accesses without byte checks (§6.3)	130	70.2	−1.4%
+ ShiftLeftImm 56 → 45 columns	130	70.4	neutral
+ shard size raised by a quarter	114	67.1	−4.7%
All four vs. evaluated baseline	114	67.1	−9.4%

byte checks on memory-instruction accesses removed 5.7% of the **LogUp**-GKR pairs and 1.4% of wall-clock time. This is less than the two-cells-per-pair model predicts because the lookup circuit’s fold layers scale with padded heights rather than pair count. Reducing a 20 M-row unit by 11 columns was below measurement noise; the 1.0s byte-check result was at its edge.

The largest single effect came from shard size. Raising it by a quarter removed 16 shards and saved 3.3s, approximately 0.2s per shard. Some shards close early on a unit’s height budget, so the reduction is smaller than the cycle fence alone suggests. GPU memory limited larger shards at this revision: at 1.5 times the baseline size, the 4.5 GiB first-layer **LogUp**-GKR buffer caused repeated trim-and-retry and a 30% slowdown; twice the baseline size did not fit in 32 GB.

E System interfaces and integration

This appendix collects the outward-facing description of the implemented system: what a deployer connects, what may be varied, how the prover and verifier are embedded, and which verification artifacts accompany a release. Section 4 fixes the corresponding system boundary.

Table 11: Interfaces of the proving and verification system. “Setup” interfaces are used once per program; “run” interfaces once per execution.

Port	When	Contents
Program image	setup	a little-endian MIPS32 ELF produced by the guest toolchain (GCC, LLVM or the Rust target); its text and initial data become the program ROM and the initial memory
Verifying key	setup	the Merkle commitment to the program ROM and the preprocessed tables (Section 6); 304 bytes, derived deterministically from the image; identifies the program in the statement
Input stream	run	the bytes the guest reads as its standard input; a witness
Hint stream	run	host-supplied data the guest reads through the hint syscall (for example the block witness); a witness, written into untouched memory and thereby bound by the memory argument
Public values	out	the digest of the committed output, the exit code, the entry and exit machine state, the address cursors of the global memory tables and the septic digest (Section 3)
Compressed proof	out	the root of the recursion tree, 603 KiB for the block of Section 11; optionally wrapped to a Groth16 proof over BN254 for on-chain verification (Section 9)
Verifier	deliverable	a verifier that checks a compressed proof on an ordinary host, small enough to embed in the consuming application; it carries the recursion-key allowlist
Accelerator interface	extension	a syscall number, an emulator hook that reads and writes guest memory at the invoking cycle, and a unit that receives the syscall tuple (arguments as 16-bit half-words plus the Linux flag) on the syscall bus and performs its memory accesses on the memory bus (Section 3)

E.1 Ports

Table 11 lists the system interfaces. Five are fixed by the statement of Section 1.1: a program image, an input stream and a hint stream go in, and a proof and its public values come out. The remaining three are supplied by a deployment: a verifying key derived once per program, a verifier that carries the key allowlist binding a recursion proof to the advertised machine, on an ordinary host or, after the wrap of Section 9, on chain, and an extension interface for guest accelerators.

E.2 Configuration space

The proving system is parameterised, and Section 11 walks through part of its configuration space. Parameters include shard size, commitment stacking height, the WHIR schedule, lookup-challenge grinding, recursion arity and guest-accelerator set. The WHIR schedule fixes committed rounds, folding factors, code rates, queries, out-of-domain samples and query grinding, thereby setting proof size and soundness accounting (Section 8). The guest-accelerator set determines precompile units and their verifying keys. Release values belong in one configuration file. A constraint-system parameter changes the affected unit keys and the recursion-key allowlist; a schedule-only parameter changes recursion-program keys but not unit keys.

E.3 Integration paths

The integrations below share the same interfaces. The common shape is a guest that hashes: it recomputes a Merkle root over data the verifier does not hold, checks an inclusion or update proof, verifies a chain of signatures or certificates, or attests a firmware image or a log. The guest is compiled by an ordinary MIPS toolchain, calls the hash accelerators of Section 3 where the volume justifies it, and the verifier checks one proof whose cost does not grow with the amount hashed.

Three specialisations are worth naming. In *block proving*, the deployed case, a coordinating process cuts a native execution into shards and ships a descriptor per shard to one worker process per GPU, which re-execute

and prove their shards and the recursion nodes above them; §11.2 describes the host budget this pipeline was engineered against. In *hybrid fault and validity proving*, one program image is adjudicated by a fault proof and certified by a validity proof without a second toolchain, because ZIREN executes the instruction set Cannon and Kona already execute [Opt23, OP 24]. In *embedded and IoT workloads*, a program compiled by a device’s own MIPS toolchain is executed by a remote prover and a gateway checks the proof, so the statement is the execution of the device’s software rather than a property of the device (§1.4). We report no measurement of the wrapped on-chain path, and Section 10.4 records two omissions found in the terminal circuits it depends on.

E.4 Verification artifacts

A release includes the verification artifacts of Section 10: the specification-derived conformance suite and Cannon reference results; the executable Lean 4 ISA model and its checks against the emulator; one extracted determinism statement per unit and opcode selector, together with the tactic and list of open statements; the soundness configuration and generated accounting report; and the defects register of Table 12. The checked-in accounting configuration uses 16 bits of lookup grinding and therefore differs from the measured zero-grinding baseline. A deployer must verify that the toolchain, runtime configuration, verifier binary, key allowlist and accounting report identify the same release. The current artifacts test ISA correspondence and close 17 of 106 determinism statements; they do not replace the open unit proofs, the trace-exhaustiveness premise or the unverified recursion machine.

F Defects surfaced by the verification flow

Table 12: Defects surfaced by the verification flow. “Theorem” means the extracted determinism theorem of the unit; “proving” means an end-to-end proof of a real block or of the test programs; “review” means a line-by-line comparison of this design against the RISC-V system it descends from, reading both implementations side by side.

Component	Defect	Found by	Resolution
Syscall unit	argument half-words not determined by the reduced argument ($2^{32} > p$)	theorem	half-words carried on the syscall bus; theorem closes
Syscall unit	result columns gated by a selector that no input pins	theorem	selector carried on the syscall bus; theorem closes
Extractor	shard and clock fields of syscall interactions dropped, leaving outputs without inputs	theorem	extractor fixed
Shape configuration	range table absent from the preprocessed heights; every shape-fixed proving test failed	proving	height registered
Standalone verifier	recursion-key Merkle root computed from the bare key list while the prover pads to the tree height	proving	verifier pads the leaves
Proving service	configured to skip the in-circuit key check, producing proofs the standalone verifier rejects	proving	service configuration
Recursion, terminal stages	the completeness flag gates every completeness predicate, including the one forcing the cross-shard sum to zero, but neither terminal circuit pinned it; an execution-prefix proof could therefore be wrapped	review	both terminal circuits assert it; the deferred program can no longer emit it set
Core, cross-shard state	the entry and exit state of a delayed-branch machine is the pair (pc, next pc), but only the first component was pinned across shards: the second was unreferenced in the recursion circuit, the prover and the verifier, so a prover could choose each shard’s continuation address	review	both components asserted in the leaf circuit and the host verifier; a forgery test rejects a shifted pair
Wrap circuit, opening	the recursive BaseFold verifier asserted only the query chain, with no sumcheck message or terminal identity, and read its query count as the smaller of the schedule’s and the list the prover supplied	review	both identities asserted in-circuit and the count taken from the schedule, a short list rejected (§8.3)
Wrap circuit	the recursion-key allowlist root was not a public input, so no external verifier could require the published one	review	exposed as a public input and checked by the standalone and on-chain verifiers
Hash parameters	partial-round count taken from the tabulation for S-box degree seven rather than three, giving less algebraic-attack margin than intended	review	schedule corrected; the measurements of Section 11 predate the change

References

- [ACFY24a] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. STIR: Reed–Solomon Proximity Testing with Fewer Queries. Cryptology ePrint Archive, Paper 2024/390, 2024.
- [ACFY24b] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. WHIR: Reed–Solomon Proximity Testing with Super-Fast Verification. Cryptology ePrint Archive, Paper 2024/1586, 2024.
- [AHIV17] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. Liger: Lightweight sublinear arguments without a trusted setup. In *ACM CCS*, 2017.
- [AST24] Arasu Arun, Srinath Setty, and Justin Thaler. Jolt: SNARKs for virtual machines via lookups. EUROCRYPT 2024; Cryptology ePrint Archive, Paper 2023/1217, 2024.
- [Axi24] Axiom. OpenVM: A performant and modular zkVM framework. <https://openvm.dev/whitepaper.pdf>, 2024.
- [Bar89] David A. Barrington. Bounded-Width Polynomial-Size Branching Programs Recognize Exactly Those Languages in NC^1 . *Journal of Computer and System Sciences*, 38(1):150–164, 1989.
- [BEG⁺94] Manuel Blum, Will Evans, Peter Gemmell, Sampath Kannan, and Moni Naor. Checking the Correctness of Memories. In *Algorithmica*, volume 12, pages 225–244, 1994.
- [BGt23] Jeremy Bruestle, Paul Gafni, and the RISC Zero Team. RISC Zero zkVM: Scalable, transparent arguments of RISC-V integrity. <https://dev.risczero.com/proof-system-in-detail.pdf>, 2023. Draft.
- [BL24] Marc Beunardeau and O(1) Labs. Research meets ecosystem: The applied cryptography team roadmap. <https://www.o1labs.org/blog/applied-cryptography-team-roadmap>, December 2024. Describes o1VM’s MIPS and RISC-V modes.
- [BSBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed–Solomon Interactive Oracle Proofs of Proximity. In *Proceedings of the 45th International Colloquium on Automata, Languages, and Programming (ICALP)*, pages 14:1–14:17, 2018.
- [BSCI⁺20] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity gaps for Reed–Solomon codes. In *FOCS*, 2020.
- [BSCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive Oracle Proofs. In *Theory of Cryptography — TCC 2016-B*, pages 31–60, 2016.
- [BSGKS20] Eli Ben-Sasson, Lior Goldberg, Swastik Kopparty, and Shubhangi Saraf. DEEP-FRI: Sampling outside the box improves soundness. In *ITCS*, 2020.
- [CJSC23] Zhigang Chen, Yuting Jiang, Xinxia Song, and Liqun Chen. A survey on zero-knowledge authentication for Internet of Things. *Electronics*, 12(5):1145, 2023.
- [Con24] Consensys. gnark: A fast zk-SNARK library in Go. <https://github.com/Consensys/gnark>, 2024.
- [CY24] Alessandro Chiesa and Eylon Yogev. Building Cryptographic Proofs from Hash Functions. <https://github.com/hash-based-snargs-book>, 2024. Textbook treatment of the BCS transformation, round-by-round soundness and state restoration.
- [Cys25] Cysic. Cysic: hardware-accelerated proof generation. <https://cysic.xyz>, 2025. Listed on the public block-proving dashboard [Eth26b].
- [DDGM23] Heini Bergsson Debes, Edlira Dushku, Thanassis Giannetsos, and Ali Marandi. ZEKRA: Zero-knowledge control-flow attestation. In *ACM ASIA Conference on Computer and Communications Security (ASIA CCS)*, 2023.

- [DP23] Benjamin E. Diamond and Jim Posen. Succinct arguments over towers of binary fields. Cryptology ePrint Archive, Paper 2023/1784, 2023.
- [DP24] Benjamin E. Diamond and Jim Posen. Polylogarithmic proofs for multilinear over binary towers. Cryptology ePrint Archive, Paper 2024/504, 2024.
- [EFG22] Liam Eagen, Dario Fiore, and Ariel Gabizon. cq: Cached quotients for fast lookups. Cryptology ePrint Archive, Paper 2022/1763, 2022.
- [EH24] Shahriar Ebrahimi and Parisa Hassanizadeh. From interaction to independence: zkSNARKs for transparent and non-interactive remote attestation. In *Network and Distributed System Security Symposium (NDSS)*, 2024. IACR ePrint 2024/1068.
- [Eth26a] Ethereum Foundation. soundcalc: security calculator for hash-based proof systems. <https://github.com/ethereum/soundcalc>, 2026.
- [Eth26b] Ethproofs. Ethproofs: real-time proving of Ethereum blocks. <https://ethproofs.org>, 2026.
- [FS87] Amos Fiat and Adi Shamir. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *Advances in Cryptology — CRYPTO '86*, pages 186–194, 1987.
- [GKR08] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. Delegating Computation: Interactive Proofs for Muggles. In *Proceedings of the 40th ACM Symposium on Theory of Computing (STOC)*, pages 113–122, 2008.
- [GKS23] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A faster version of the Poseidon hash function. In *AFRICACRYPT*, 2023.
- [GLS⁺21] Alexander Golovnev, Jonathan Lee, Srinath Setty, Justin Thaler, and Riad Wahby. Brakedown: Linear-time and field-agnostic SNARKs for R1CS. Cryptology ePrint Archive, Paper 2021/1043, 2021.
- [GW20] Ariel Gabizon and Zachary J. Williamson. plookup: A simplified polynomial protocol for lookup tables. Cryptology ePrint Archive, Paper 2020/315, 2020. <https://eprint.iacr.org/2020/315>.
- [Hab22] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Paper 2022/1530, 2022.
- [Har25] David Harold. MIPS at 40. Jon Peddie Research, <https://www.jonpeddie.com/news/mips-at-40/>, January 2025.
- [HJR⁺25] Tamir Hemo, Kevin Jue, Eugene Rabinovich, Gyumin Roh, and Ron D. Rothblum. Jagged Polynomial Commitments (or: How to Stack Multilinears). Cryptology ePrint Archive, Paper 2025/917, 2025.
- [HLP24] Ulrich Haböck, David Levit, and Shahar Papini. Circle STARKs. Cryptology ePrint Archive, Paper 2024/278, 2024.
- [HR18] Justin Holmgren and Ron Rothblum. Delegating Computations with (almost) Minimal Time and Space Overhead. Electronic Colloquium on Computational Complexity, Report TR18-161, 2018.
- [LFKN92] Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. Algebraic Methods for Interactive Proof Systems. *Journal of the ACM*, 39(4):859–868, 1992.
- [Lit24] Lita Foundation. Valida ISA specification, version 1.0: A zk-optimized instruction set architecture. <https://github.com/lita-xyz/valida-releases>, 2024.

- [LZZ⁺24] Tianyi Liu, Zhenfei Zhang, Yuncong Zhang, Wenqing Hu, and Ye Zhang. Ceno: Non-uniform, segment and parallel zero-knowledge virtual machine. Cryptology ePrint Archive, Paper 2024/387, 2024.
- [Mat25] Matter Labs. ZKsync Airbender: A RISC-V zkVM over Mersenne-31. <https://github.com/matter-labs/zksync-airbender>, 2025.
- [MIP26] MIPS Tech LLC. IP processors. <https://mips.com/products/hardware/>, 2026. States “over 8.5B MIPS-based devices shipped”. Accessed September 2026.
- [MT21] Dimitris Mouris and Nektarios Georgios Tsoutsos. Zilch: A framework for deploying transparent zero-knowledge proofs. *IEEE Transactions on Information Forensics and Security*, 2021. IACR ePrint 2020/1155, <https://eprint.iacr.org/2020/1155>.
- [Nex24] Nexus Labs. Nexus 1.0: Enabling verifiable computation. <https://whitepaper.nexus.xyz/>, 2024.
- [OP 24] OP Labs. Kona: A rust implementation of the OP Stack fault-proof program. <https://github.com/op-rs/kona>, 2024.
- [Ope26] OpenVM Project. Formal verification of the OpenVM RISC-V, Keccak, and SHA-2 extensions. <https://github.com/openvm-org/openvm-fv>, 2026. Lean proofs extracted from OpenVM v2.0.0.
- [Opt23] Optimism Foundation. Cannon: A MIPS interpreter for Optimism fault proofs. <https://github.com/ethereum-optimism/optimism>, 2023.
- [PCW⁺23] Shankara Pailoor, Yanju Chen, Franklyn Wang, Clara Rodríguez, Jacob Van Geffen, Jason Morton, Michael Chu, Brian Gu, Yu Feng, and Işıl Dillig. Automated detection of under-constrained circuits in zero-knowledge proofs. In *PLDI*, 2023.
- [PH23] Shahar Papini and Ulrich Haböck. Improving logarithmic derivative lookups using GKR. Cryptology ePrint Archive, Paper 2023/1284, 2023.
- [Pol24] Polygon Zero and contributors. Plonky3: A toolkit for STARKs over small fields. <https://github.com/Plonky3/Plonky3>, 2024.
- [QV15] Nguyen Anh Quynh and Dang Hoang Vu. Unicorn: The ultimate CPU emulator. Black Hat USA; <https://www.unicorn-engine.org/>, 2015.
- [RIS24] RISC Zero. STARK soundness calculator. <https://github.com/risc0/risc0/tree/main/risc0/zkp/src/prove/soundness.rs>, 2024.
- [SD21] Xavier Salleras and Vanesa Daza. ZPiE: Zero-knowledge proofs in embedded systems. *Mathematics*, 9(20):2569, 2021. IACR ePrint 2021/1382.
- [ST25] Srinath Setty and Justin Thaler. Twist and Shout: faster memory checking arguments via one-hot addressing and increments. Cryptology ePrint Archive, Paper 2025/105, 2025. Adopted by Jolt from v0.2.0 in place of Lasso and Spice.
- [Sta21] StarkWare Industries. Cairo: A Turing-complete STARK-friendly CPU architecture. <https://www.cairo-lang.org>, 2021.
- [Sta23] StarkWare. ethSTARK documentation—version 1.2. Cryptology ePrint Archive, Paper 2021/582, 2023.
- [STW24] Srinath Setty, Justin Thaler, and Riad Wahby. Unlocking the lookup singularity with Lasso. EUROCRYPT 2024; Cryptology ePrint Archive, Paper 2023/1216, 2024.
- [Suc24] Succinct Labs. SP1: A performant, open-source, contributor-friendly zkVM. <https://github.com/succinctlabs/sp1>, 2024. Accessed 2026.

- [Suc25a] Succinct Labs. SP1 Hypercube: Real-time proving of Ethereum blocks. <https://blog.succinct.xyz/sp1-hypercube/>, 2025. Technical report and open-source implementation.
- [Suc25b] Succinct Labs. sp1-jit: Just-in-time RISC-V executor for SP1. <https://github.com/succinctlabs/sp1>, 2025. Component of the SP1 repository.
- [Suc26] Succinct Labs. SP1 security model. <https://docs.succinct.xyz/docs/sp1/security/security-model>, 2026. Retrieved 2026-09-24.
- [Sup25] Supranational. sppark: Zero-knowledge template library for accelerating SNARK provers. <https://github.com/supranational/sppark>, 2025. Apache-2.0; the inline-PTX Montgomery field primitives used here are `ff/mont_t.cuh` and `ff/ptx.cuh` by Dag Arne Osvik. Retrieved 2026-09-25.
- [The25] The ZKM Project. Ziren: A zero-knowledge MIPS virtual machine. <https://docs.zkm.io>, 2025.
- [Tur21] Jim Turley. Wait, what? MIPS becomes RISC-V. *EE Journal*, March 2021. <https://www.eejournal.com/article/wait-what-mips-becomes-risc-v/>.
- [ZCF23] Hadas Zeilberger, Binyi Chen, and Ben Fisch. BaseFold: Efficient Field-Agnostic Polynomial Commitment Schemes from Foldable Codes. Cryptology ePrint Archive, Paper 2023/1705, 2023.
- [Zis25] ZisK Project. ZisK: A high-performance zkVM. <https://github.com/0xPolygonHermez/zisk>, 2025.
- [ZKM24] ZKM. zkMIPS: A high-performance zero-knowledge proof system for the MIPS architecture. <https://github.com/zkMIPS/zkMIPS>, 2024.